Machine Learning

17/04/2024

Course Overview

Machine learning is a branch of computer science that uses algorithms to imitate the way in which humans learn. It uses statistical methods to train algorithms and make predictions. The accuracy of these predictions improves over time.

This course covers the basic concepts and techniques of Machine Learning from both theoretical and practical perspective

Topics

Introduction to Artificial Intelligence and Machine Learning

Data Prepossessing:

- Feature Scaling.
- Missing Data.
- Dummy Variable.
- Imbalanced Data.

Feature Engineering.

- Backward Elimination.
- Forward Elimination.
- Model Validation

Supervised Learning:

- Linear Regression.
- Logistic Regression.
- Naive Bays.
- Regression and Classification Metrics, Confusion Matrix, ROC and AUC.

Ensemble Learning

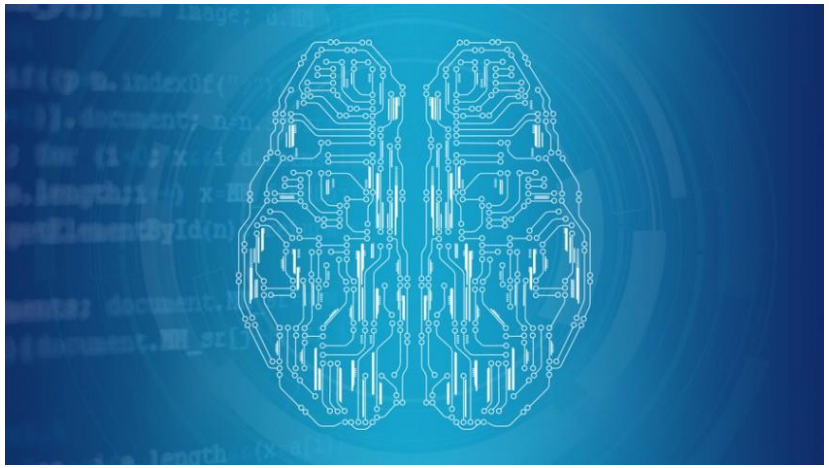
- Decision Tree.
- Random Forest.
- Gradient Boost, XG Boost and LGBM

Unsupervised learning

- K-means Clustering.
- Hieratical Clustering.
- Gaussian Mixture Model GMM.
- Principal Component Analysis PCA.

What & Why ?

- Grew out of work in AI
- New Capability for computers
- Adaptive thinking of computers
- The core of the future technologies



Definition

- Arthur Samuel (1959). Machine Learning: Field of study that gives computers the ability to learn without being explicitly programmed.



- Tom Mitchell (1998). Well-posed Learning Problem: A computer program is said to learn from experience E with respect to some task T and some performance measure P, if its performance on T, as measured by P, improves with experience E.



WHY ?

- Flood of available data (especially with the advent of the Internet) “2.5 quintillion / day (10^30)”
- Increasing computational power
- Growing progress in available algorithms and theory developed by researchers
- Increasing support from industries

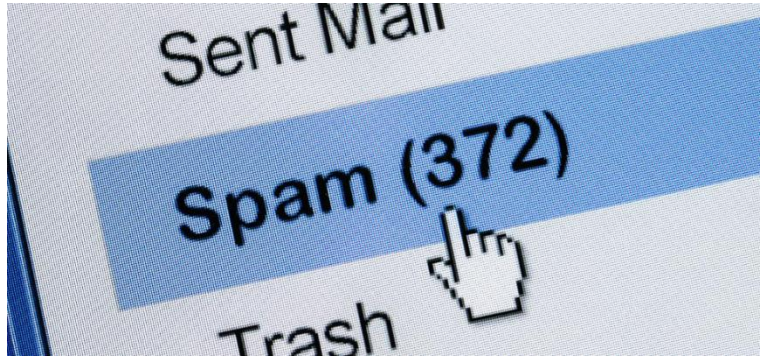
HOW ?

E.g. Spam mails program

T >> Classifying Emails as Spam or not

E>> Watching you label emails as spam or not spam

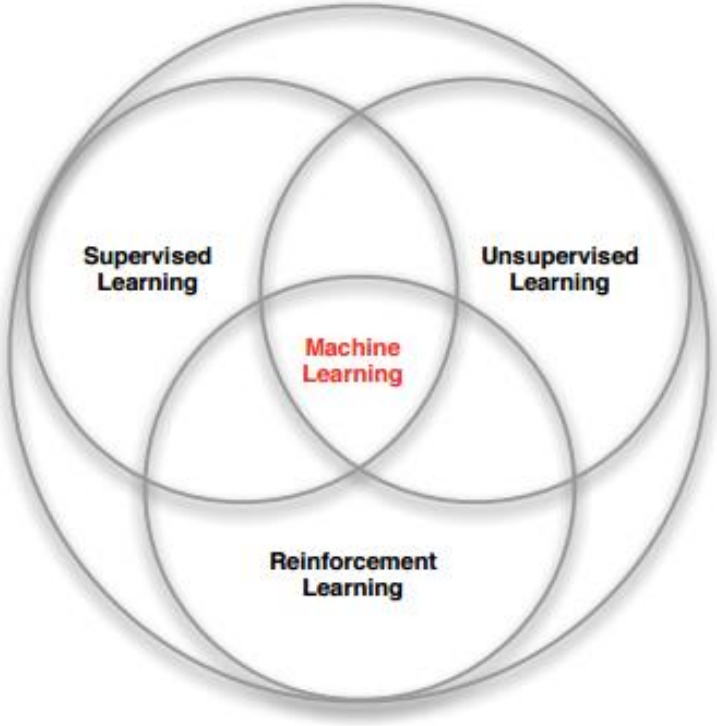
P>> Number of emails correctly classified as Spam / Not Spam

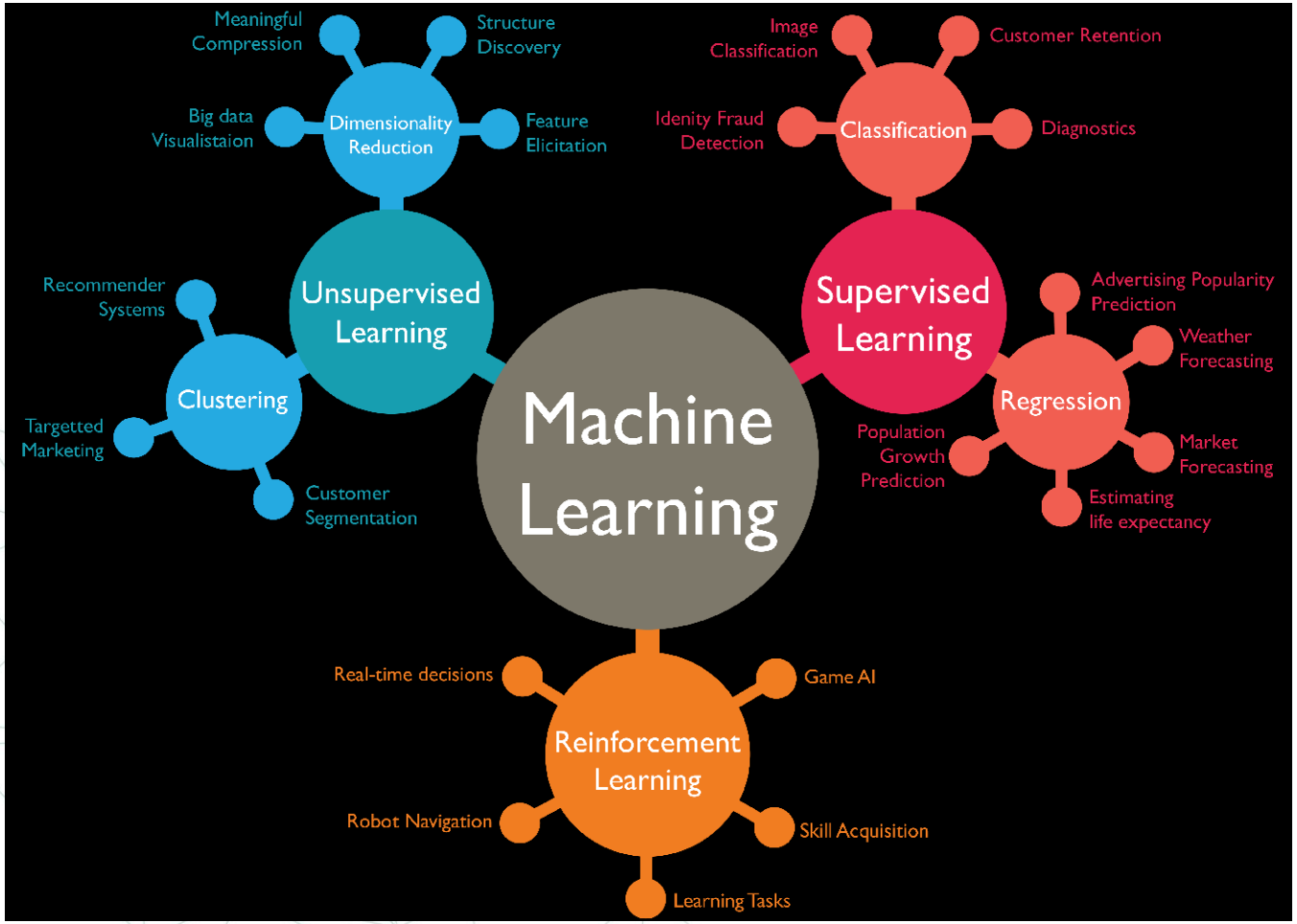


Machine Learning Algorithms

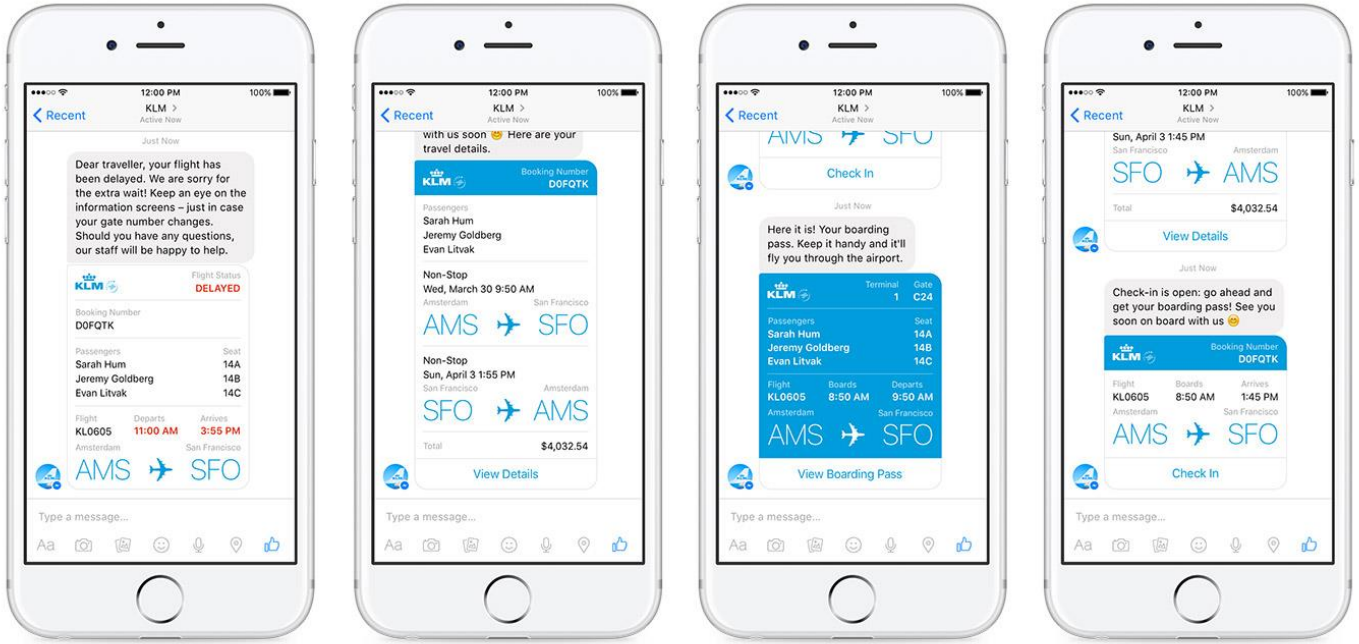
- Supervised Learning
- Unsupervised Learning
- Reinforcement Learning

- Others: Recommender Systems, … etc.




Machine Learning
Unsupervised Learning
Dimensionality Reduction
Meaningful Compression
Structure Discovery
Big data Visualistaion
Feature Elicitation
Clustering
Recommender Systems
Targetted Marketing
Customer Segmentation
Supervised Learning
Classification
Image Classification
Customer Retention
Idenity Fraud Detection
Diagnostics
Regression
Advertising Popularity Prediction
Weather Forecasting
Population Growth Prediction
Market Forecasting
Estimating life expectancy
Reinforcement Learning
Real-time decisions
Game AI
Robot Navigation
Skill Acquisition
Learning Tasks

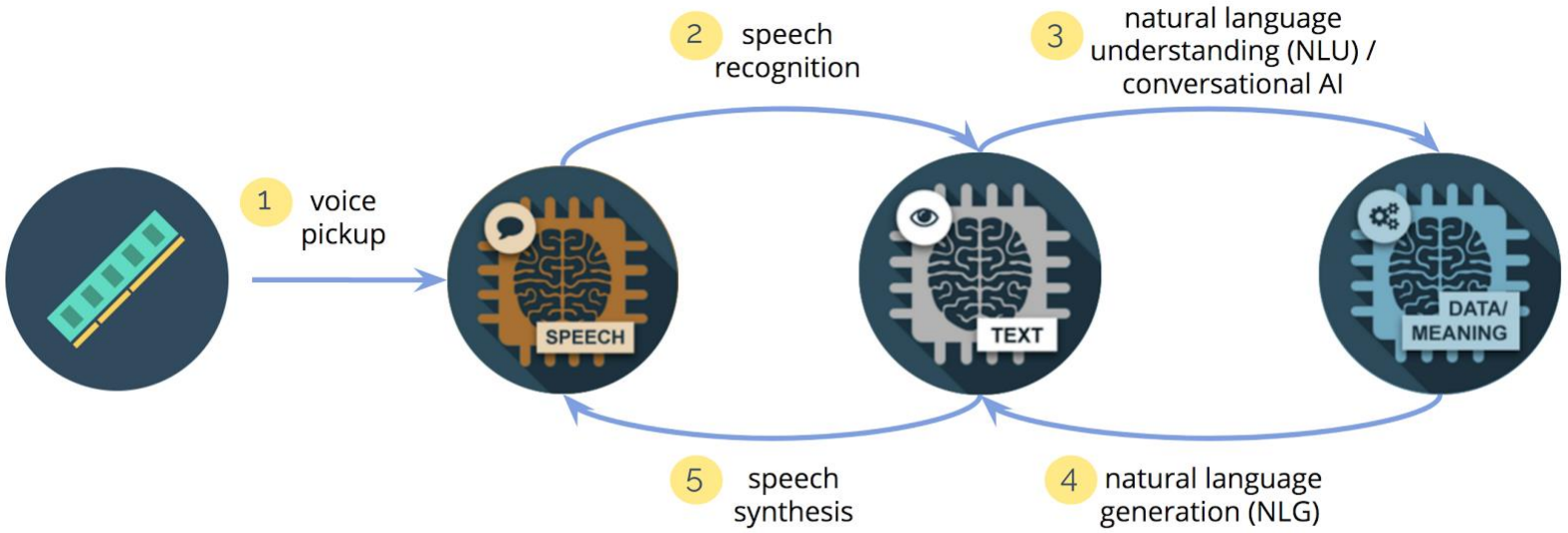
Facebook – Chatbot Army



Google – ‘Machines That Dream’



Baidu – The Future of Voice Search



* There may be substitutions and more complicated functionalities within NLU/conversational AI in the future.

Others

- Data Security
- Personal Security
- Financial Trading
- Healthcare
- Marketing Personalization
- Fraud Detection
- Recommendations
- Online Search
- Natural Language Processing (NLP)
- Smart Cars

The astounding athletic power of quad copters



The astounding athletic power of quadcopters Raffaello D'Andrea.mp4

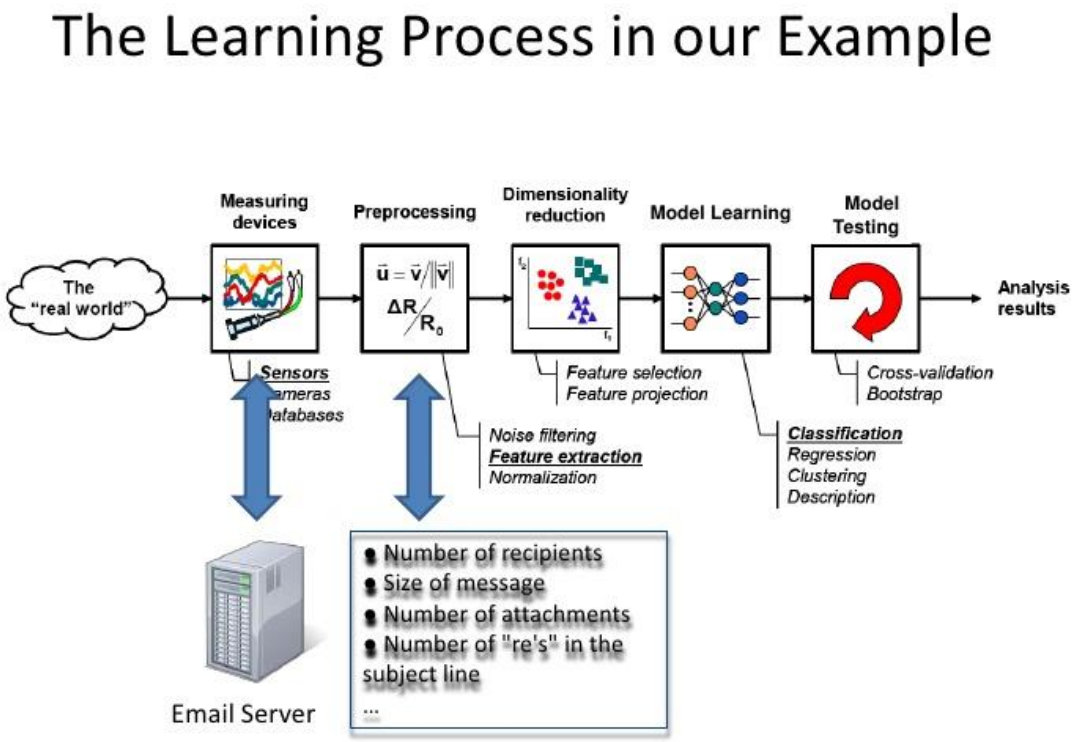
Supervised learning

- classification is seen as supervised learning from examples.

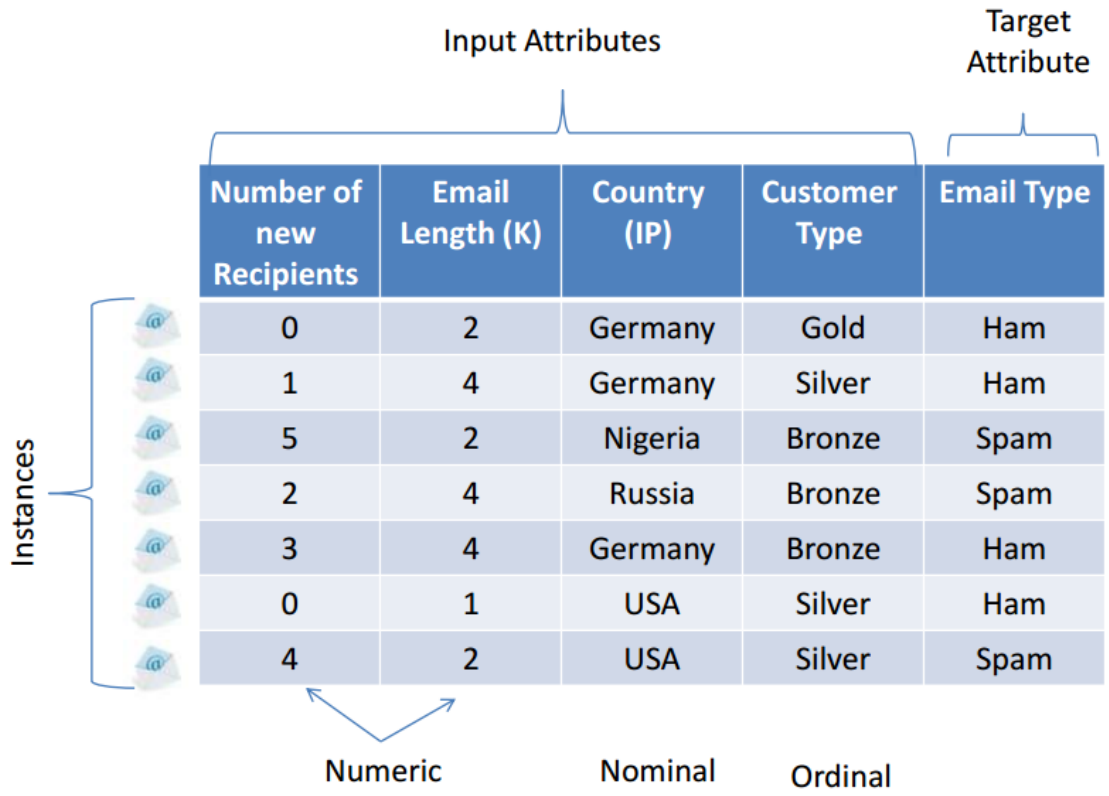
- Supervision: The data (observations, measurements, etc.) are labeled with pre-defined classes. It is like that a “teacher” gives the classes (supervision).

- Test data are classified into these classes too.

The learning process



Data set

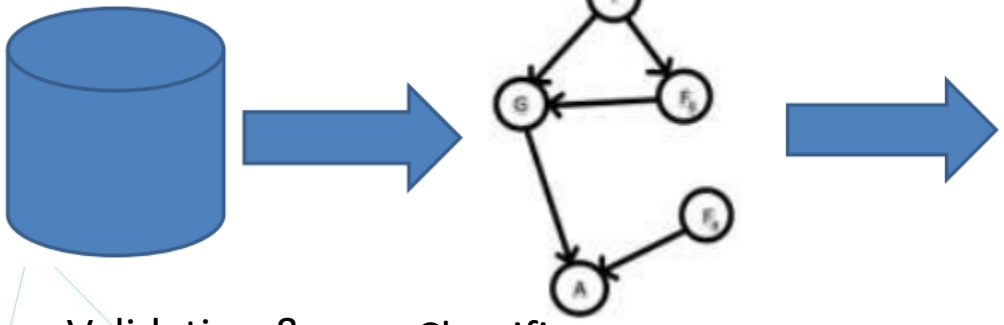


Number of new Recipients	Email Length (K)	Country (IP)	Customer Type	Email Type
0	2	Germany	Gold	Ham
1	4	Germany	Silver	Ham
5	2	Nigeria	Bronze	Spam
2	4	Russia	Bronze	Spam
3	4	Germany	Bronze	Ham
0	1	USA	Silver	Ham
4	2	USA	Silver	Spam

Model Learning



Model testing

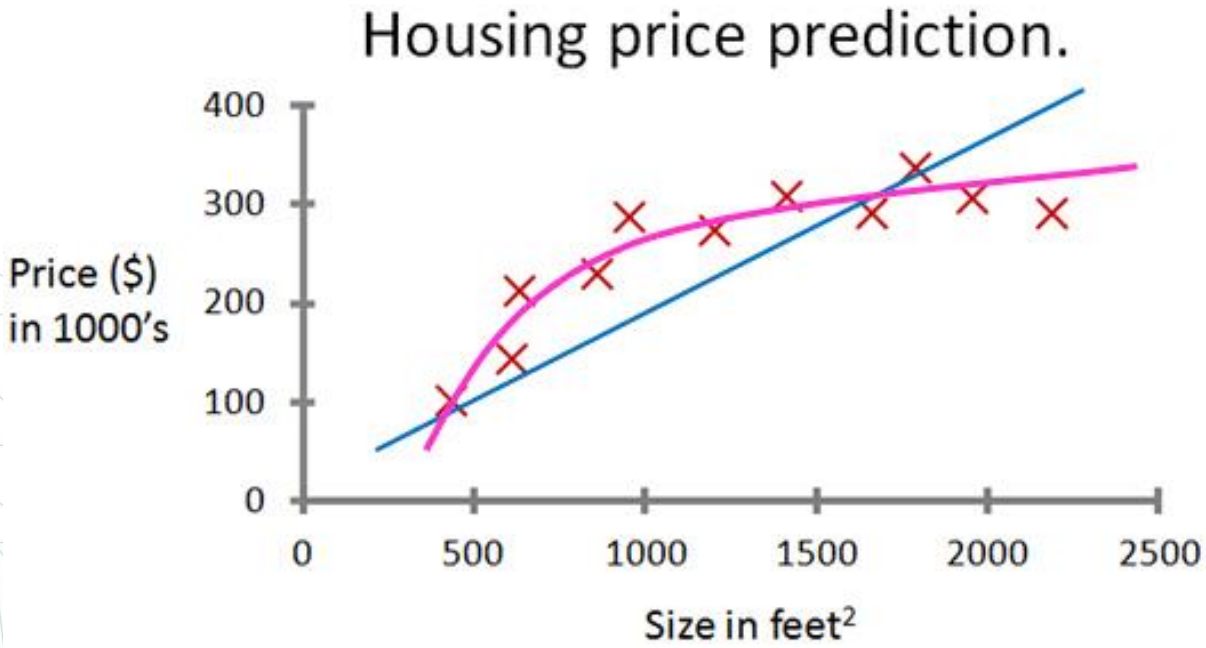


Cross Validation &
Testing Set

Classifier
Classification Model

Evaluation

Prediction / regression



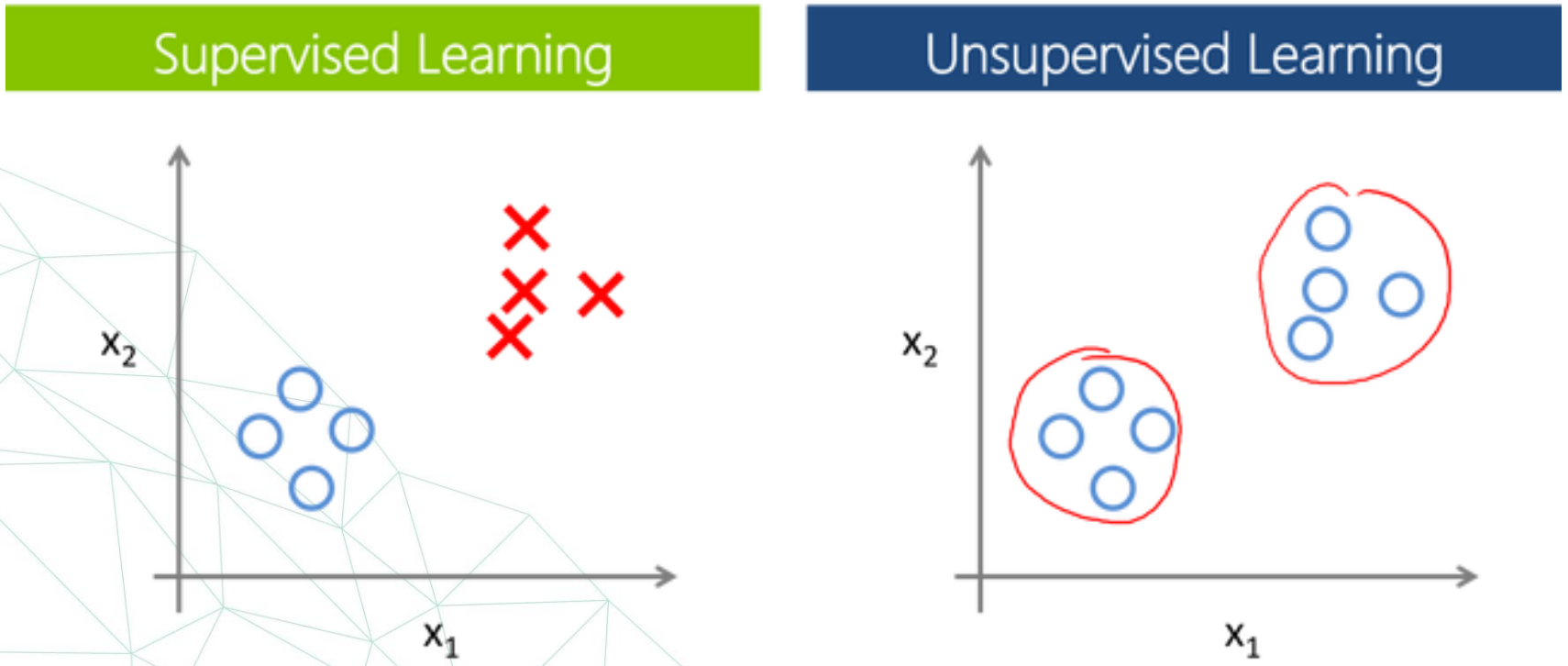
Question

- We have a hospital application for the suspects of a cancer, they get the data of the hospital and contains many features and attributes, to have an efficient system to decide whether the tumor is malignant or benign. What would be the type of the problem?

- We have a hyper market which contains many types of products, according to the data of the market we want to know how the profits of the market for the next month. What would be the type of the problem?

Unsupervised learning

- given the data can you derive a certain group / cluster ?

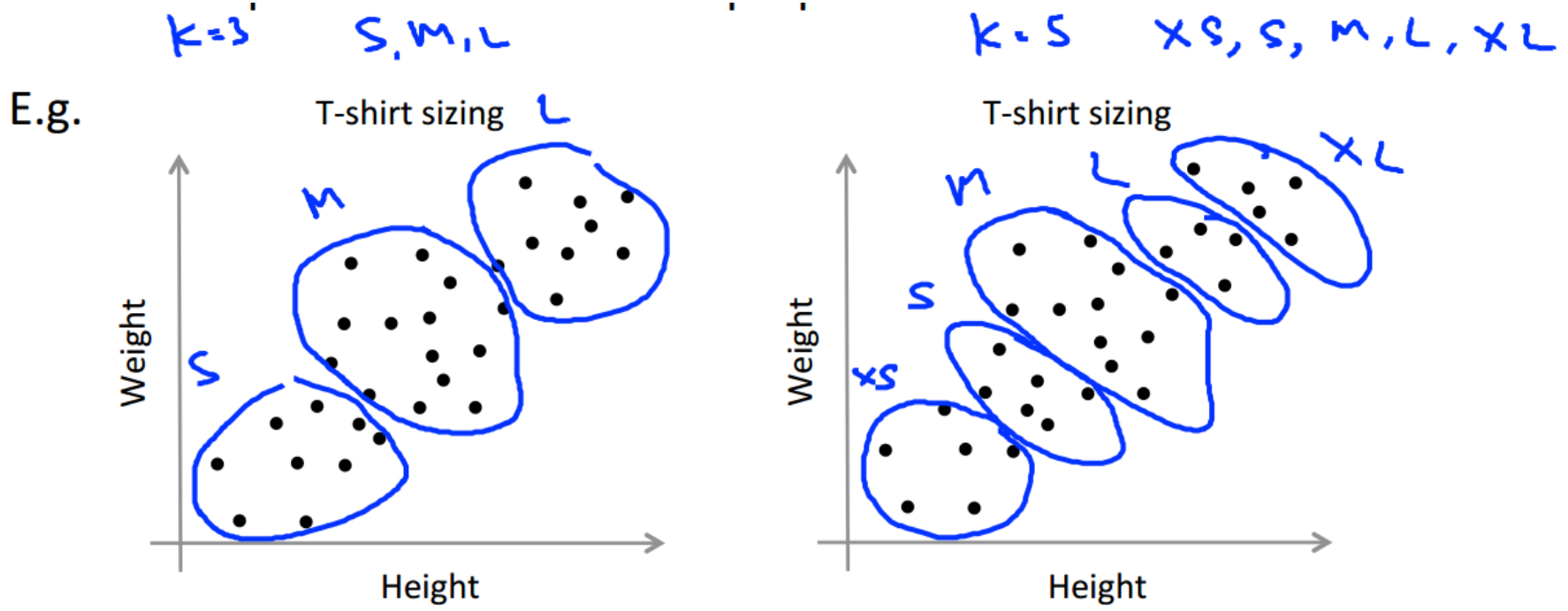


Unsupervised learning

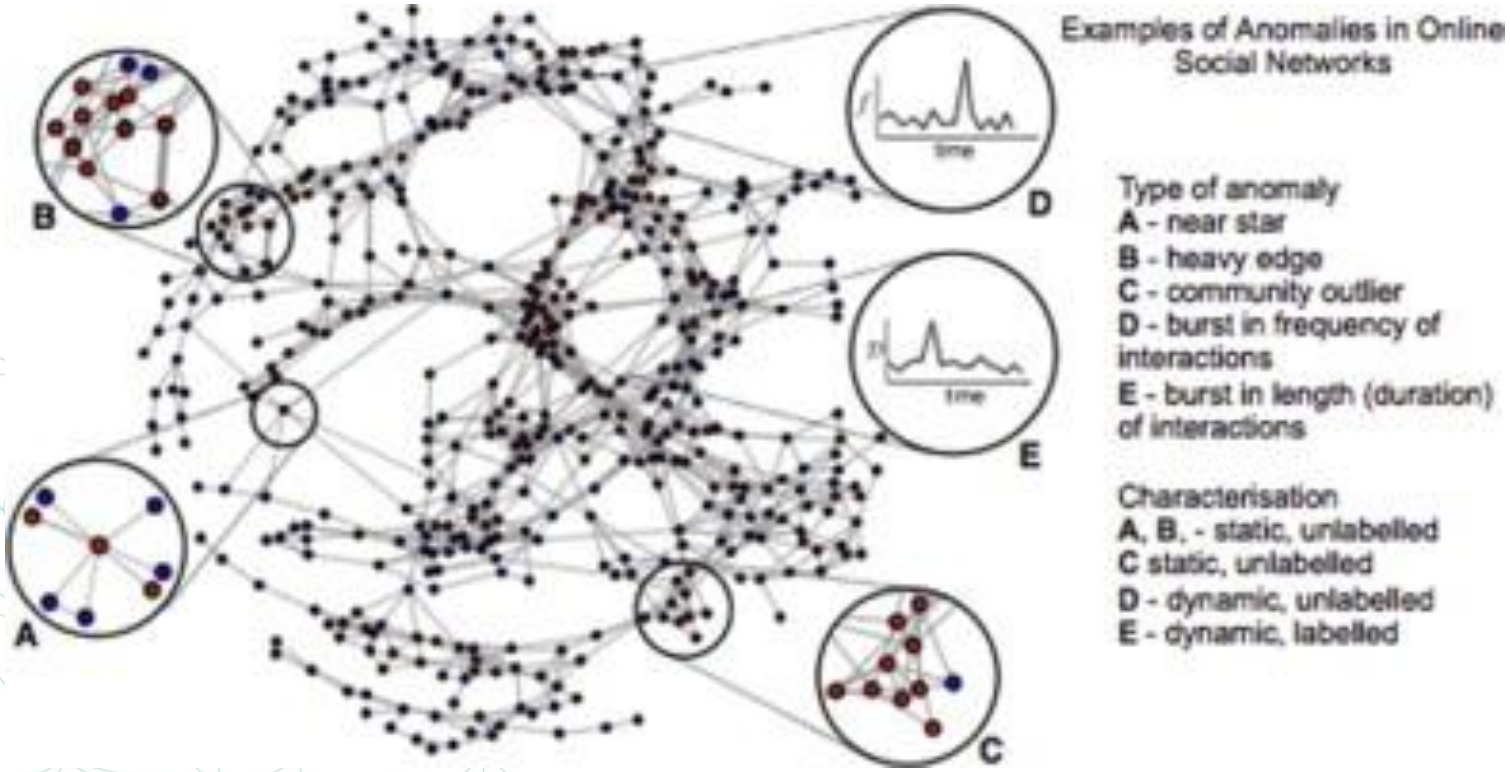
- studies how systems can learn to represent particular input patterns in a way that reflects the statistical structure of the overall collection of input patterns.
- It is likely to be much more common in the brain than supervised learning.
- Examples:
- Clustering.
- Blind signal separation.
- Self- organising maps
- Etc.

segmentation

E.g.

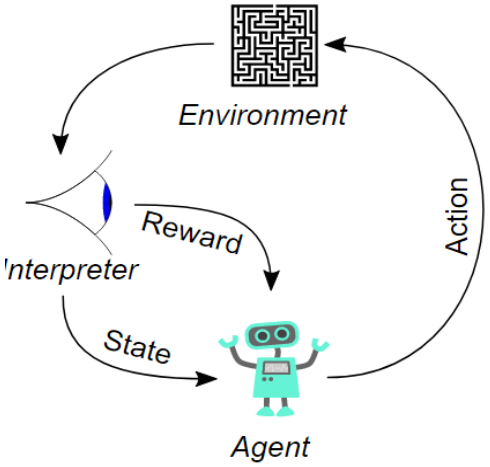


Social network analysis



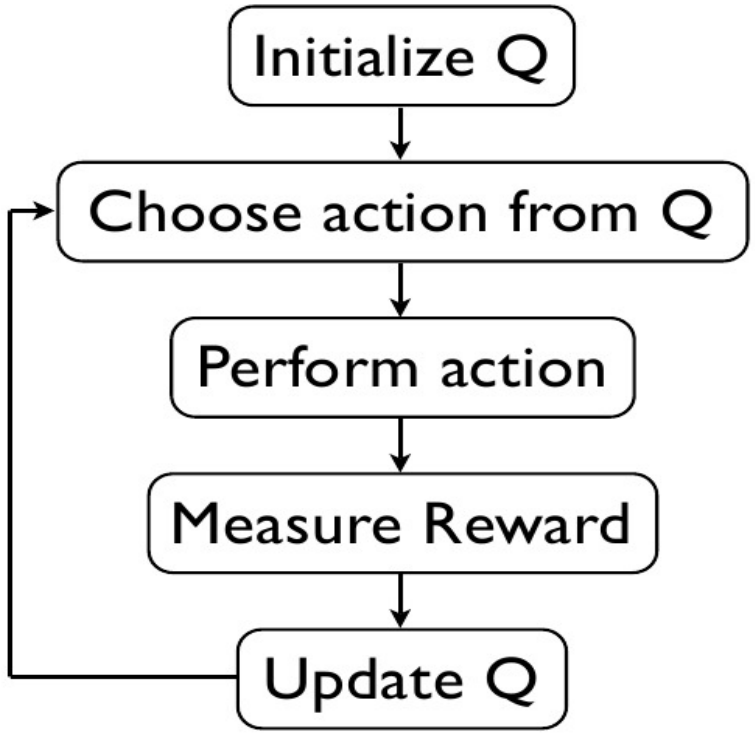
Reinforcement Learning

- Learning from interaction with an environment to achieve some long-term goal that is related to the state of the environment
- The goal is defined by reward signal, which must be maximized
- Agent must be able to partially/fully sense the environment state and take actions to influence the environment state
- The state is typically described with a feature-vector



Q learning

$$R = \begin{array}{c} \text{State} \\ 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \text{Action} \\ \begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 \end{array} \end{array} \begin{bmatrix} -1 & -1 & -1 & -1 & 0 & -1 \\ -1 & -1 & -1 & 0 & -1 & 100 \\ -1 & -1 & -1 & 0 & -1 & -1 \\ -1 & 0 & 0 & -1 & 0 & -1 \\ 0 & -1 & -1 & 0 & -1 & 100 \\ -1 & 0 & -1 & -1 & 0 & 100 \end{bmatrix}$$



Q&A

Questions and answers

Data Preprocessing

What is Data Preprocessing?

- It is a data mining technique that transforms raw data into an understandable format. Raw data(real world data) is always incomplete and that data cannot be sent through a model. That would cause certain errors. That is why we need to preprocess data before sending through a model.

Steps in Data Preprocessing

1. Import libraries
2. Read data
3. Checking for missing values
4. Checking for categorical data
5. Standardize the data
6. PCA transformation
7. Data splitting

Import Data

- As main libraries, I am using Pandas, NumPy and time;
  Pandas: Use for data manipulation and data analysis.
  NumPy: a fundamental package for scientific computing with Python.
- As for the visualization I am using **Matplotlib** and Seaborn.
  For the data preprocessing techniques and algorithms, I used **Scikit-learn** libraries.

Read Data

- You can find more details on dataset here: https://www.kaggle.com/

Checking for missing values

- If there are no missing values found in the dataset, I didn’t use any missing values handling techniques.

Checking for categorical data

- If there is any categorical data we should convert it to numerical ones

Checking the data distribution

- Check it through plotting onto **Seaborn** library

Standardizing the data

- If the dataset do contain only numerical input variables which are the result of a PCA transformation. Features V1, V2, … V28 are the principal components obtained with PCA, the only features which have not been transformed with PCA may be a private data. So PCA is affected by scale so we need to scale the features in the data before applying PCA. For the scaling I am using Scikit- lean's Standard Scaler(). In order to fit to the scaler, the data should be reshaped within -1 and 1.

Dimensionality Reduction

- PCA (Principal Component Analysis) mainly using to reduce the size of the feature space while retaining as much of the information as possible. In here all the features transformed into 2 features using PCA.

Confusion Matrix

- A confusion matrix is a table that is often used to describe the performance of a classification model (or “classifier”) on a set of test data for which the true values are known. Scikit-learn provides facility to calculate confusion matrix using the confusion_matrix method.

Data Preprocessing

Purpose of data preprocessing

Get data overview: understand the data formats and overall structure in which the data is stored. Also, find the properties of data like mean, median, standard quantiles and standard deviation. These details can help identify irregularities in the data.

Identify missing data: missing data is common in most real-world datasets. It can disrupt true data patterns, and even lead to more data loss when entire rows and columns are removed because of a few missing cells in the dataset.

Identify outliers or anomalous data: some data points fall far out of the predominant data patterns. These points are outliers, and might need to be discarded to get predictions with higher accuracies, unless the primary purpose of the algorithm is to detect anomalies.

Remove Inconsistencies: just like missing values, real-world data also has multiple inconsistencies like incorrect spellings, incorrectly populated columns and rows (eg.: salary populated in gender column), duplicated data and much more. Sometimes, these inconsistencies can be treated through automation, but most often they need a manual check-up.

Imports First

Pandas
Numpy
Matplotlib
SKlearn

Read Data

Read the data from a CSV

```
# pandas used to load data
import pandas as pd
from pandas import read_csv
filename = 'movie.csv'
data = read_csv(filename)
data.head(5)
```

	movieId	title	genres
0	1	Toy Story (1995)	NaN
1	2	Moses(1990)	Adventure|Children|Fantasy
2	3	Grumpier Old Men (1995)	Comedy|Romance
3	2	Moses(1990)	Adventure|Children|Fantasy
4	4	MoTech(2019)	Comedy|Drama|Romance

Missing Data (numerical)

Remove the lines where data are missing

```
In [46]: df1 = pd.DataFrame(dict)
         df1.dropna(axis = 0)#along the row
```

Out[46]:

	First Score	Second Score	Third Score	Fourth Score
0	100.0	30.0	52.0	60
3	95.0	56.0	98.0	65

Missing Data (numerical)

```
In [35]: fourthdf=df.fillna(0)
         fourthdf
```

Out[35]:

	Name	Sex	Age	Ticket	Fare
20	Fynney, Mr. Joseph J	male	35.0	239865	26.0000
21	Beesley, Mr. Lawrence	male	34.0	248698	13.0000
22	McGowan, Miss. Anna "Annie"	female	15.0	330923	8.0292
23	Sloper, Mr. William Thompson	male	28.0	113788	35.5000
24	Palsson, Miss. Torborg Danira	female	8.0	349909	21.0750
25	Asplund, Mrs. Carl Oscar (Selma Augusta Emilia...	female	38.0	347077	31.3875
26	Emir, Mr. Farred Chehab	male	0.0	2631	7.2250
27	Fortune, Mr. Charles Alexander	male	19.0	19950	263.0000
28	O'Dwyer, Miss. Ellen "Nellie"	female	0.0	330959	7.8792
29	Todoroff, Mr. Lalio	male	0.0	349216	7.8958

Missing Data (numerical)

```
In [158]: def replace_nan(value):
              if pd.isnull(value):
                  return df['Rating'].mean()
              else:
                  return value

In [161]: df['Rating'] = df['Rating'].apply(replace_nan)

In [162]: df
```

Out[162]:

	Player	Rating	Team	Grade
0	mike	94.0	Pandas	Great
1	John	95.0	Pandas	Great
2	luke	93.0	Numpy	Excellent
3	Tom	94.0	Django	Excellent

Missing Data (numerical)

Build a model with other features to predict the missing values

Categorical Data

Label Encoding

That system of labeling implies a hierarchical value to the data that could affect your model. 3 has a higher value than 0, but cat is not (necessarily…) greater than moose.

```
# creating an instance of labelencoder
labelencoder = LabelEncoder()

# Assigning numerical values and storing it in another column called "State_N"
covid19["State_N"] = labelencoder.fit_transform(covid19["State"] )

# Display Dataframe
covid19
```

	State	Confimed	Deaths	Recovered	State_N
0	Maharashtra	284281	11194	158140	3
1	Tamil Nadu	156369	2236	107416	4
2	Delhi	118645	3545	97693	0
3	Karnataka	51422	1032	19729	2
4	Gujarat	45481	2089	32103	1
5	Uttar Pradesh	43441	1046	26675	5

Categorical Data

Create Dummy variable
For both use One Hot Encoder

Scikit-Learn
OneHotEncoder

```
encoder = OneHotEncoder()
df_fruit_encoded = pd.DataFrame(encoder.fit_transform(df[['fruit']]).todense(),
                                columns=encoder.get_feature_names())
df_fruit_encoded
```

	x0_apple	x0_banana	x0_orange
0	1.0	0.0	0.0
1	1.0	0.0	0.0
2	0.0	1.0	0.0
3	0.0	0.0	1.0
4	0.0	1.0	0.0
5	1.0	0.0	0.0

Pandas
get_dummies

```
pd.get_dummies(df['size'])
```

) just 1 line!

	large	medium	small
0	1	0	0
1	0	1	0
2	0	0	1
3	1	0	0
4	0	1	0
5	0	0	1

Categorical Data

Binary Encoding
Base N Encoding

```
Temp_dict = { 'Cold' : 1,
              'Warm' : 2,
              'Hot' : 3,
              'Very Hot' :4}
df['Temp_Ordinal'] = df.Temperature.map(Temp_dict)
df
```

	Temperature	Color	Target	Temp_Ordinal
0	Hot	Red	1	3
1	Cold	Yellow	1	1
2	Very Hot	Blue	1	4
3	Warm	Blue	0	2
4	Hot	Red	1	3
5	Warm	Yellow	0	2
6	Warm	Red	1	2
7	Hot	Yellow	0	3
8	Hot	Yellow	1	3
9	Cold	Yellow	1	1

Categorical Data

Hashing

```
df = pd.DataFrame({'a':['asds','asdds','asdsadsdas'],'b':['asewer','werwer','tyutyuty']})
df
Out[42]:
            a         b
0        asds    asewer
1       asdds    werwer
2  asdsadsdas  tyutyuty
In [43]:

df.applymap(hash)

Out[43]:
                     a                    b
0  4065519673257264805  7631381377676870653
1 -2144933431774646974 -6124472830212927118
2 -3091042543719078458 -1784823178011532358
```

Categorical Data

Bayesian encoders

These encoders borrow information from the target variable and map them to the categories in a column. This technique is of great use when the number of categories is significantly high. In such cases, if classic encoders are used, it will increase the dimensionality many-fold and trouble the model unnecessarily. When the number of features is extremely high, a problem called the curse of dimensionality reduces the efficiency of machine learning models since they’re still not adept enough to handle large volumes of features.

Standard Scaler	$\frac{x_i - \text{mean}(x)}{\text{stdev}(x)}$
MinMax Scaler	$\frac{x_i - \min(x)}{\max(x) - \min(x)}$
Robust Scaler	$\frac{x_i - Q_1(x)}{Q_3(x) - Q_1(x)}$

Feature Scaling

Well, Imagine our data. We have one column with animal ages from 4–17 and we have animal worth that ranges from $48,000-$83,000. Not only is the worth column made up of much higher numbers than the age column, but the variables also cover a much wider range of data. That means that the Euclidean distance will be dominated by worth and will wind up dominating the age data.

Feature Scaling

Standard Scaler

$$z = \frac{x - \mu}{\sigma}$$

```
from sklearn.preprocessing import StandardScaler
```

```
sta = StandardScaler()
```

Feature Scaling

Mean Normalization

```
# main normalization
mean = df['columns'].mean()
max_ = df['columns'].max()
min_ = df['columns'].min()

df['columns'] = [(x-mean)/(max_-min_) for x in df['columns']]
```

Feature Scaling

MinMax Scaler

```
from sklearn.preprocessing import MinMaxScaler
mms = MinMaxScaler()
X_train_norm = mms.fit_transform(X_train)
X_test_norm = mms.transform(X_test)
```

```
X_train_norm[0]
```

```
array([ 0.72043011,  0.20378151,  0.53763441,  0.30927835,  0.33695652,
        0.54316547,  0.73700306,  0.25      ,  0.40189873,  0.24068768,
        0.48717949,  1.        ,  0.5854251 ])
```

```
X_test_norm[0]
```

```
array([ 0.72849462,  0.16386555,  0.47849462,  0.29896907,  0.52173913,
        0.53956835,  0.74311927,  0.13461538,  0.37974684,  0.4364852 ,
        0.32478632,  0.70695971,  0.60566802])
```

Feature Scaling

Max-Abs Scaler

```
from sklearn.preprocessing import RobustScaler
#create scaler
scaler = RobustScaler()

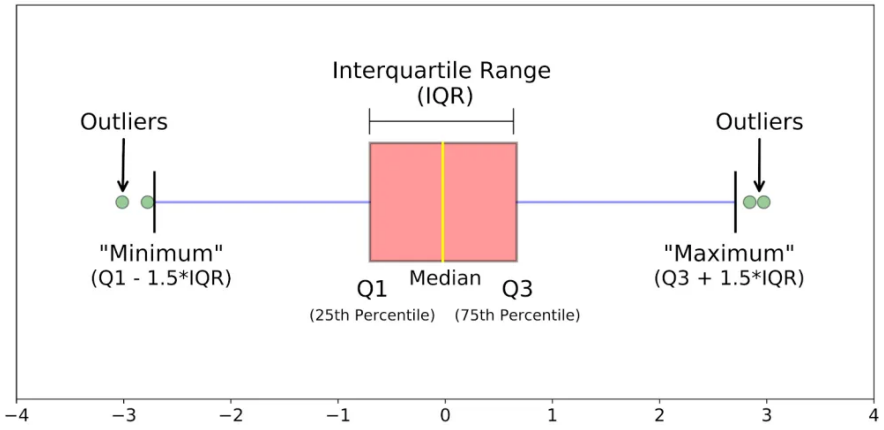
#apply scaler
datasetSinOutlierRobust = scaler.fit_transform(np.array(datasetSinOutlier).reshape(-1,1))

#print data
print (datasetSinOutlierRobust.ravel())

#plot
plt.plot(x, datasetSinOutlierRobust, '-', linewidth=2, color="orange")
```

Outliers treatment

Outliers are data points that do not conform with the predominant pattern observed in the data. They can cause disruptions in the predictions by taking the calculations off the actual pattern.



Outliers treatment

Outliers can be detected and treated with the help of box-plots. Box plots are used to identify the median, interquartile ranges and outliers. To remove the outliers, the maximum and minimum range needs to be noted, and the variable can be filtered accordingly.

Outliers treatment

```
import seaborn as sns
sns.boxplot(x=boston_df['DIS'])
```

```
from scipy import stats
import numpy as np


z = np.abs(stats.zscore(boston_df))
print(z)
```

Feature creation

New features can be created from raw features. For example, if two features called ‘total time’ and ‘total distance’ are available, you can create the feature of speed. This gives a new perspective to the model, which can now detect a logical relation between speed and the target variable. I.e. Date of birth → Age

Feature creation

https://scikit-learn.org/stable/modules/classes.html#module-sklearn.decomposition

Dimensionality reduction

“ The objective of variable selection is three-fold: improving the prediction performance of the predictors, providing faster and more cost-effective predictors, and providing a better understanding of the underlying process that generated the data.”

Dimensionality reduction

PCA
LDA
SVD

Train Test Split

Use SKlearn train test split

References

https://neptune.ai/blog/data-preprocessing-guide

رواد مصر الرقمية

وزارة الاتصـــــالات
وتكنولوجيا المعلومات

Q&A

Questions and answers

Linear regression

Prediction / regression



Data set

Training set of housing prices (Portland, OR)

Size in feet² (x)	Price ($) in 1000's (y)
2104	460
1416	232
1534	315
852	178
...	...

Notation:

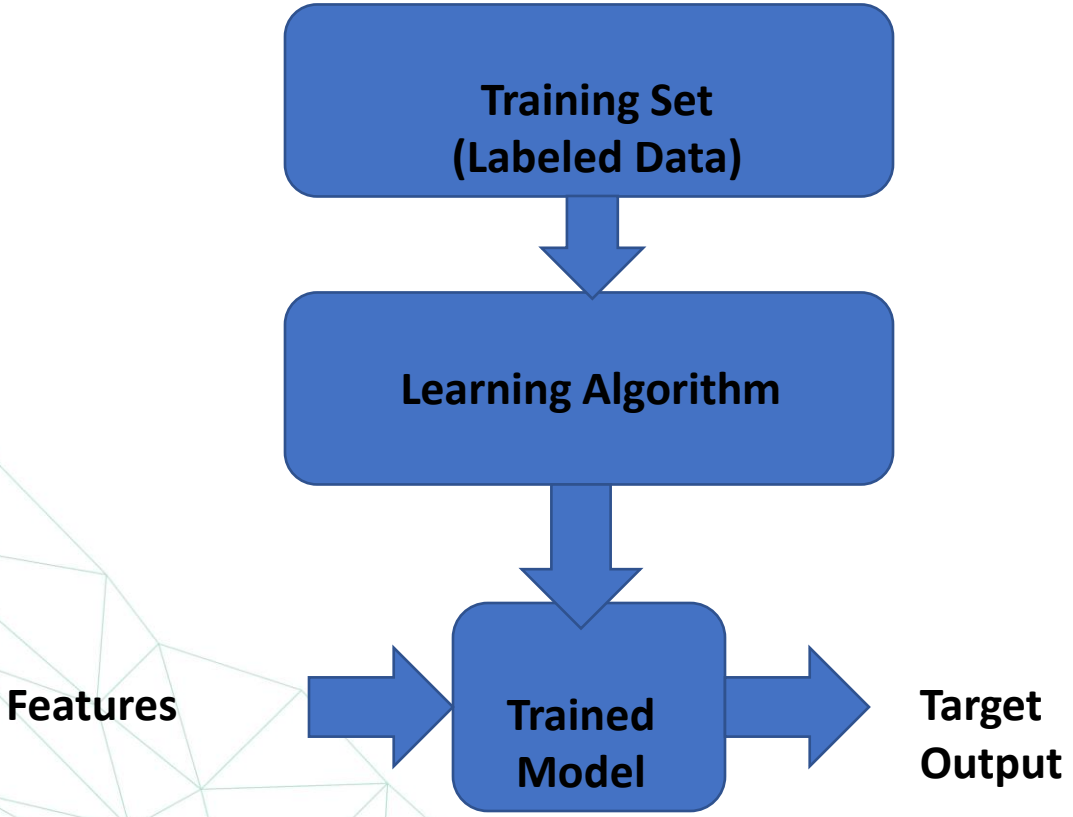
- **m** = Number of training examples
- **x**'s = "input" variable / features
- **y**'s = "output" variable / "target" variable

Notations

- $X^{(i)}$ → i_{th} feature example in the data
- $Y^{(i)}$ → i_{th} target example in the data

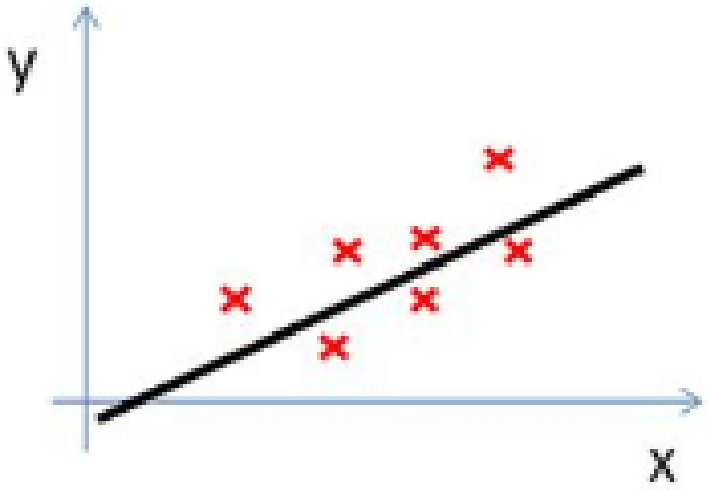
- X(1) = 2104
- X(2) = 1416
- Y(4) = 178

Supervised learning model



Linear regression

- For one variable
  - $h_\Theta(x) = \Theta_0 + \Theta_1 x$



Hypothesis

$$h_\Theta(x) = \Theta_0 + \Theta_1 x$$

Θ_i parameters → How to choose it wisely?

What if?

$\Theta_0 = 0.5$	|	$\Theta_0 = 0$	|	$\Theta_0 = 0.5$
$\Theta_1 = 0$	|	$\Theta_1 = 0.5$	|	$\Theta_1 = 1$

optimization

- At last we want to draw a line that fits the data most
- Means we choose Θ_0 , Θ_1 so that $h_\Theta(x)$ is close to Y for the
- Training example (X,Y)

- Minimize $_{\Theta 0\ ,\ \Theta 1}$ $\frac{1}{2m}\sum_{i=1}^{m}(h_\Theta(x^i) - y^i)^2$

$$h_\Theta(x) = \Theta_0 + \Theta_1 x$$

Find me the Θ0 , Θ1 that leads to the minimum

Cost function

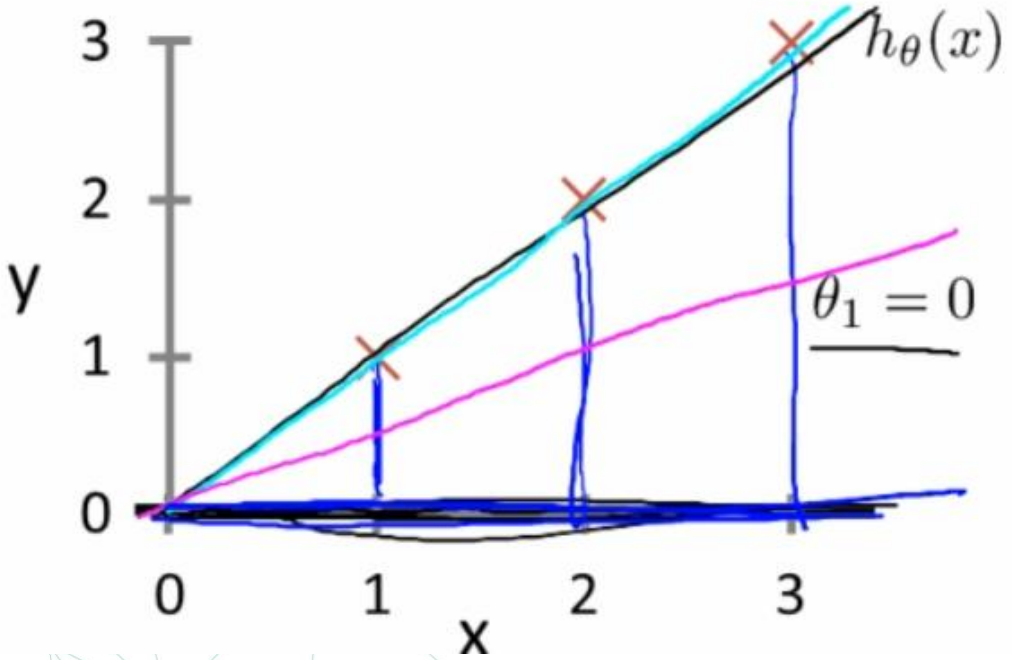
- $J(\Theta 0 , \Theta 1) = \frac{1}{2m} \sum_{i=1}^{m} (h_\Theta(xi) - y^i)^2$

Sum Squared Error

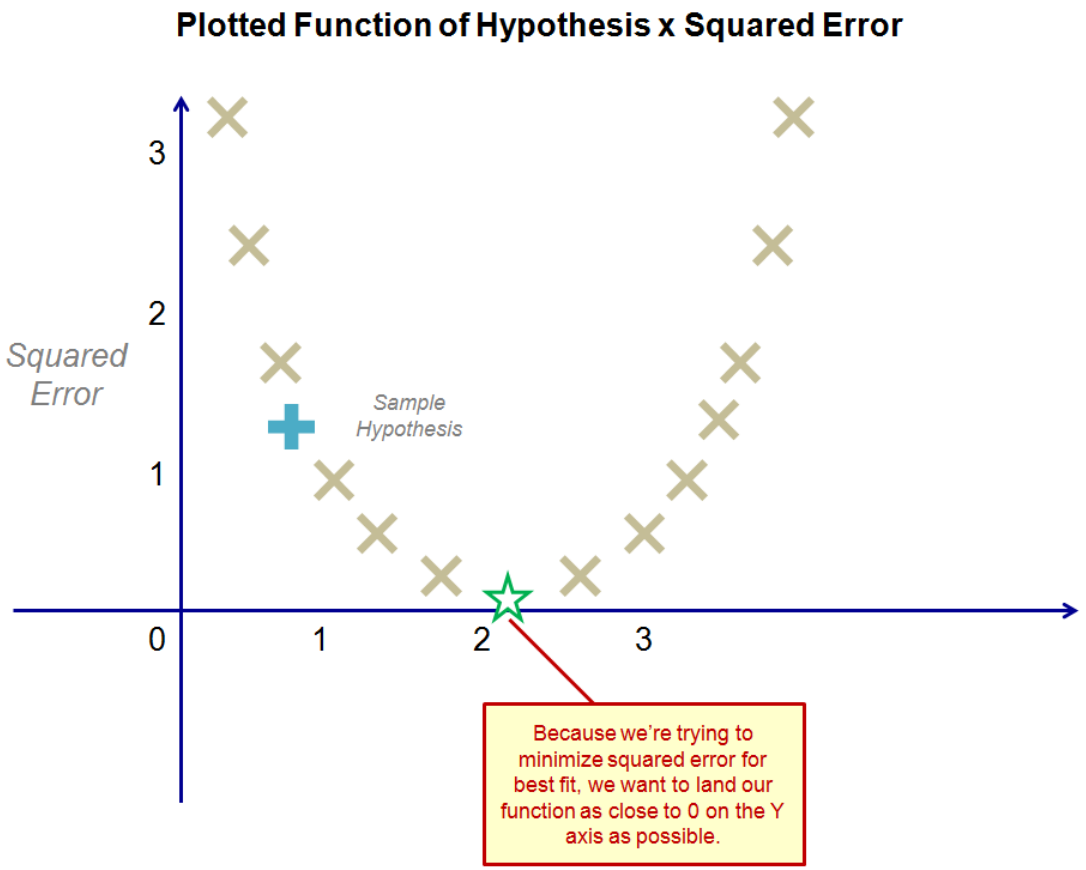
Cost function

- Let $\Theta_0 = 0$, $h_\Theta(x) = \Theta_1 x$

(for fixed θ_1, this is a function of x)

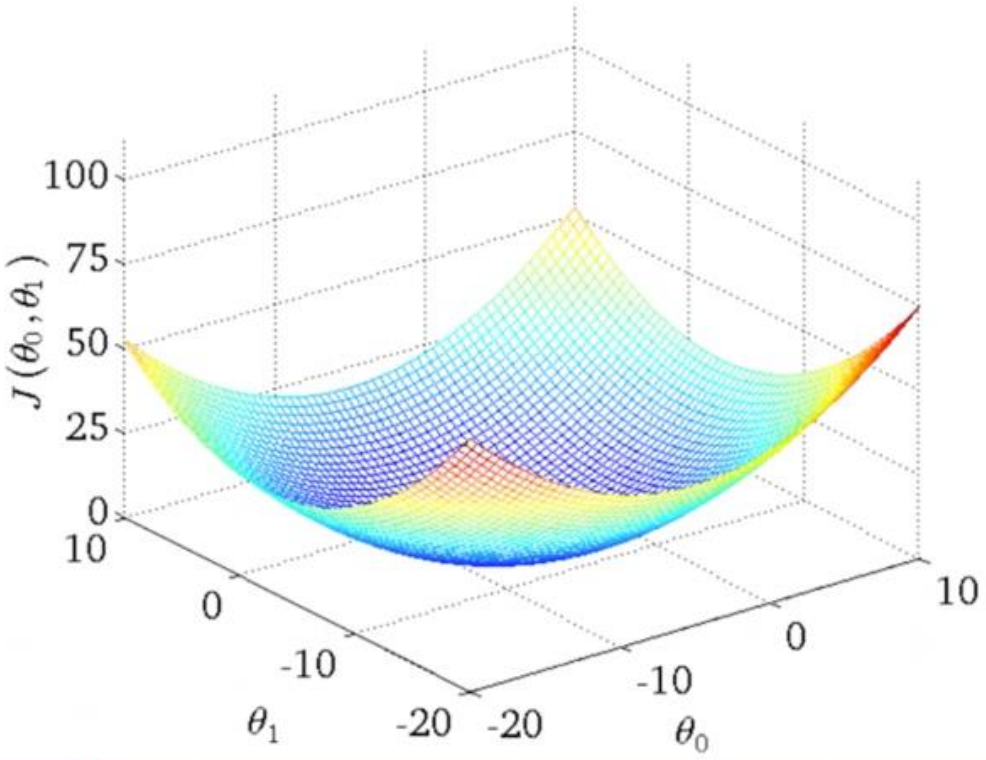


Cost function



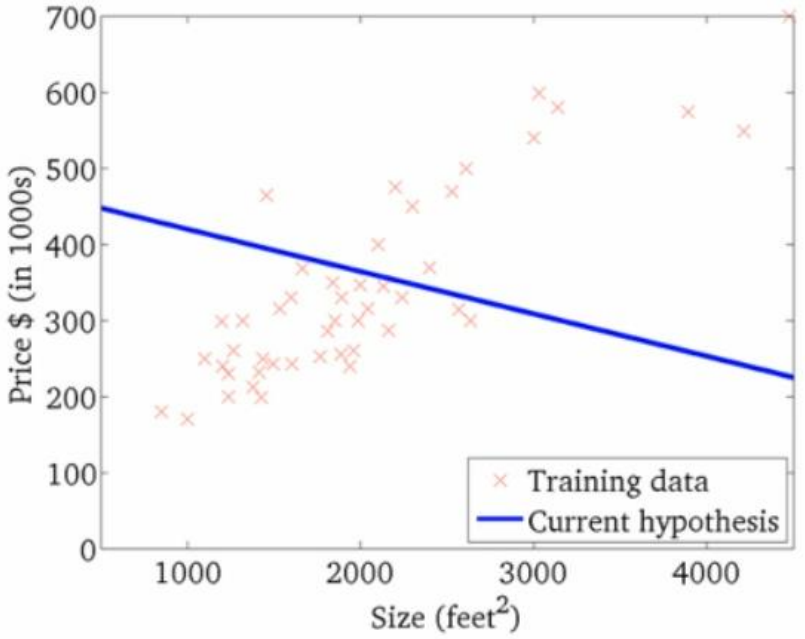
Cost function

If we used Θ0 , Θ1 then
The cost function would
Be represented as
bawl shape
3 dimensional
as shown in figure

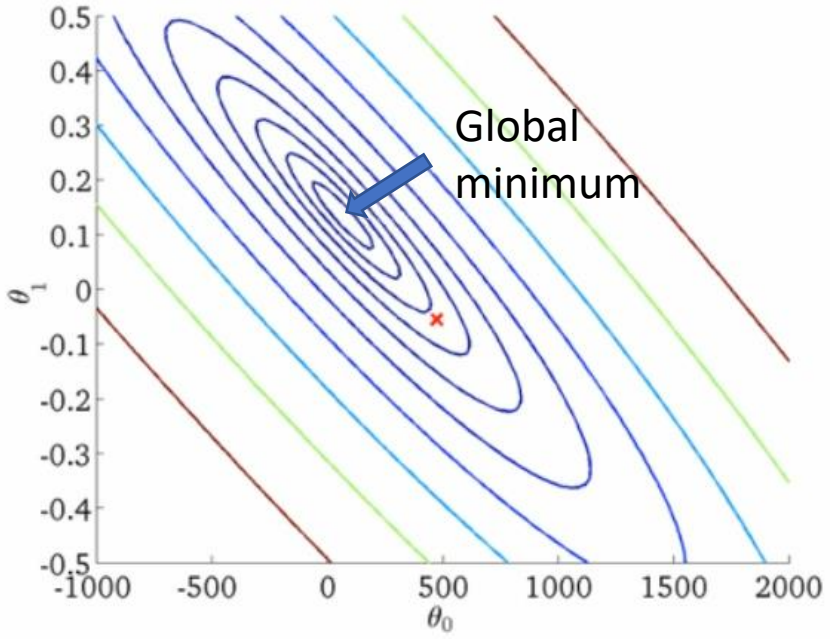


Cost function

- (for fixed θ_0, θ_1, this is a function of x)

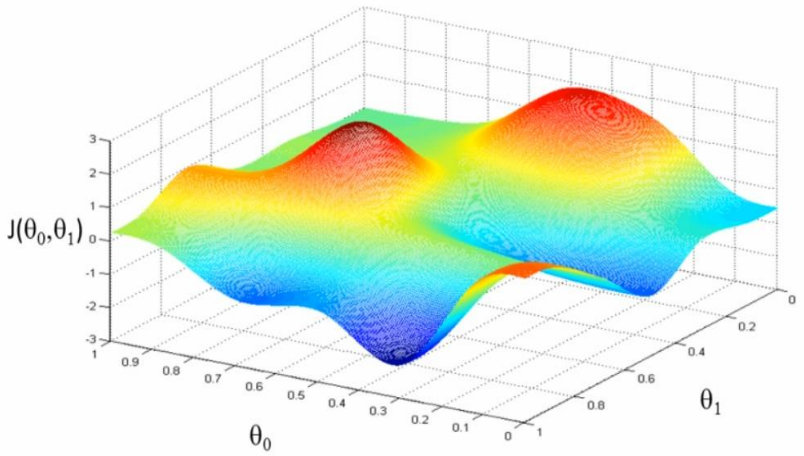


(function of the parameters θ_0, θ_1)

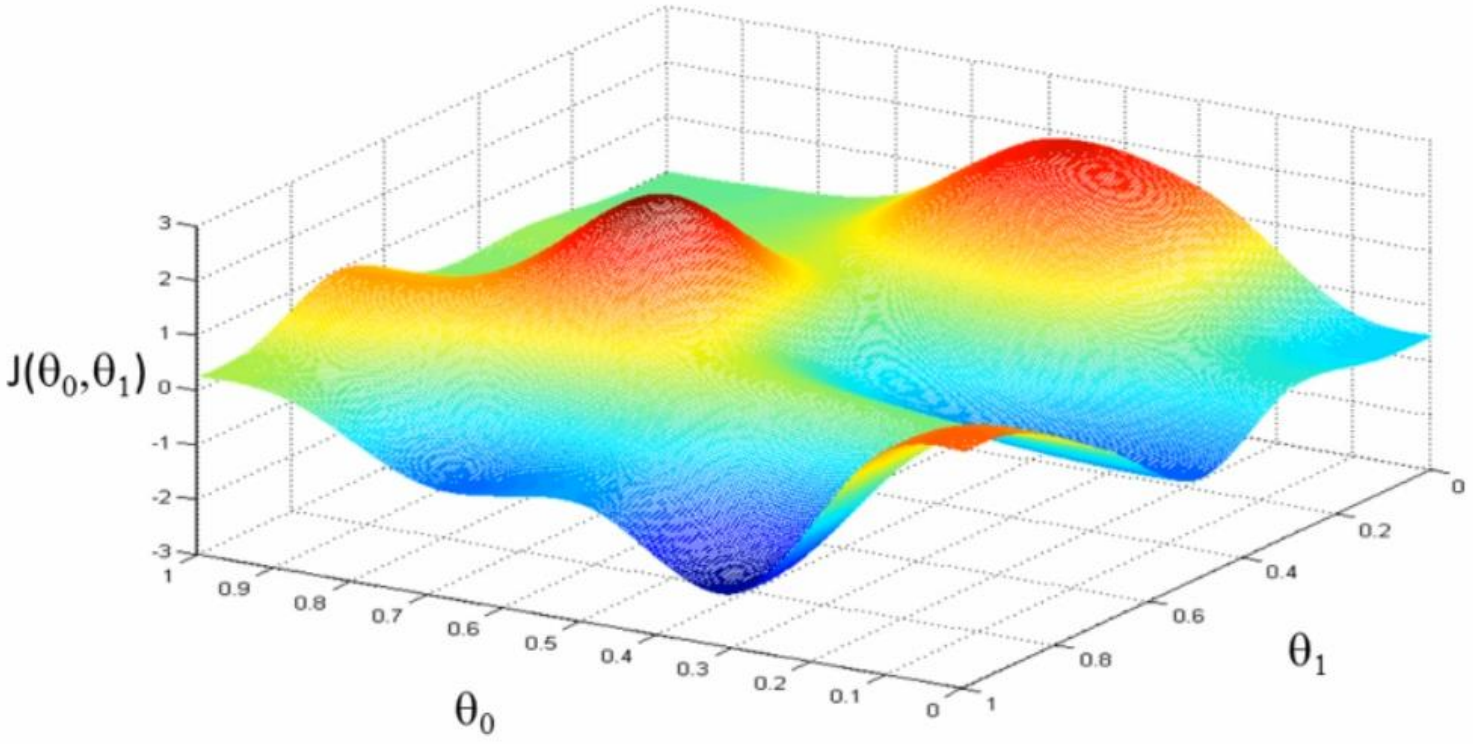


Gradient descent

- First, we start off in a random place (means we start with random Θ0 , Θ1 then calculate the cost function)
- Gradually change the Θ0 , Θ1 in order to reduce J(Θ0 , Θ1)
- Keep moving with a certain step size until reach to the global optimum “minimum”



Gradient descent



Gradient descent Algorithm

Keep repeating until convergence

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta).$$

Feature j parameter

Step size

Derivative Of the cost function

In simple steps

$$\text{temp0} := \theta_0 - \alpha \frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1)$$
$$\text{temp1} := \theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1)$$
$$\theta_0 := \text{temp0}$$
$$\theta_1 := \text{temp1}$$

Note : Must be Simultaneously Updated as shown

Gradient descent Algorithm

- Incorrect implementation :

Incorrect:

$$\text{temp0} := \theta_0 - \alpha \frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1)$$
$$\theta_0 := \text{temp0}$$
$$\text{temp1} := \theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1)$$
$$\theta_1 := \text{temp1}$$



Gradient descent Algorithm

- The cost function derivation

$$
\begin{aligned}
\frac{\partial}{\partial \theta_j} J(\theta) &= \frac{\partial}{\partial \theta_j} \frac{1}{2} \left(h_\theta(x) - y\right)^2 \\
&= 2 \cdot \frac{1}{2} \left(h_\theta(x) - y\right) \cdot \frac{\partial}{\partial \theta_j} \left(h_\theta(x) - y\right) \\
&= \left(h_\theta(x) - y\right) \cdot \frac{\partial}{\partial \theta_j} \left(\sum_{i=0}^{n} \theta_i x_i - y \right) \\
&= \left(h_\theta(x) - y\right) x_j
\end{aligned}
$$

$$
\theta_j := \theta_j + \alpha \left(y^{(i)} - h_\theta(x^{(i)}) \right) x_j^{(i)} .
$$

Gradient descent problems

- What if α is too small ?

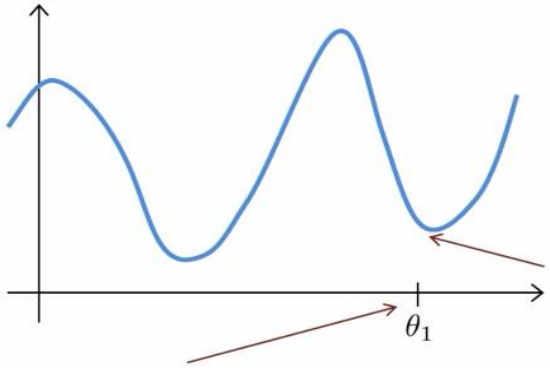
May be too slow

- What if α is too large ?

May overshoot and may fail to converge

- What if there is many local optimum?

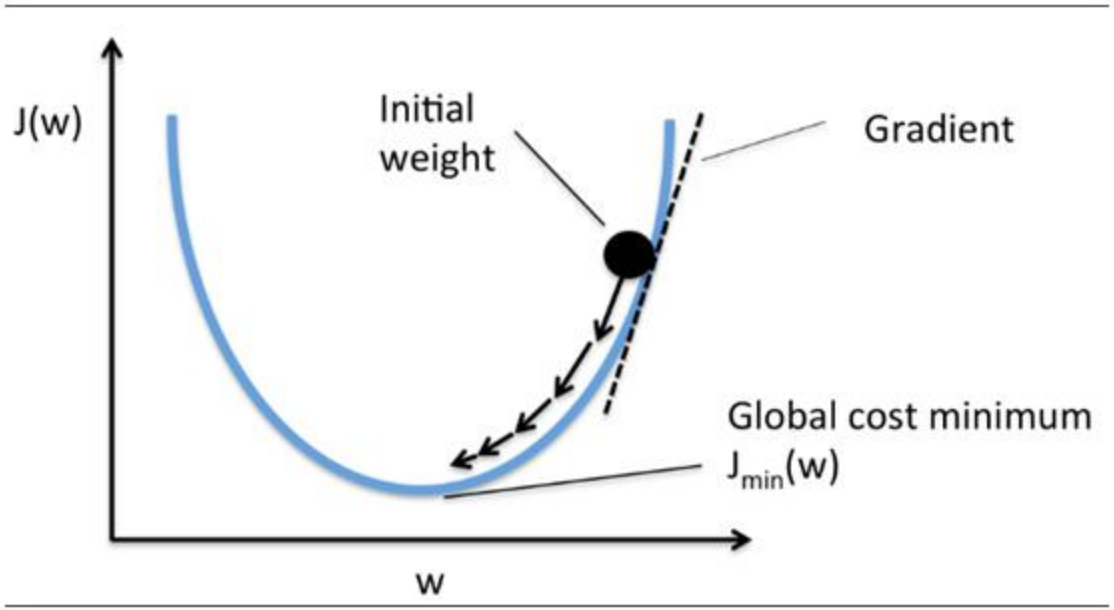
May get stuck in one of the local optimum



Gradient descent

Gradient descent may converge to a local minimum even with the Learning rate fixed

As we approach a local minimum , gradient descent will automatically take smaller steps



Linear regression with multiple variables

Single feature

Size (feet²)	Price ($1000)
x	y
2104	460
1416	232
1534	315
852	178
...	...

$$h_\theta(x) = \theta_0 + \theta_1 x$$

Multiple features

- here we have four features to predict price

Size (feet^2)	Number of bedrooms	Number of floors	Age of home (years)	Price ($1000)
2104	5	1	45	460
1416	3	2	40	232
1534	3	2	30	315
852	2	1	36	178
...	...	...	...	...

Hypothesis

Previously: $h_\theta(x) = \theta_0 + \theta_1 x$

- with multiple features :
- Where n is the number of features

$$h_\theta(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

Hypothesis summarize

Hypothesis: $h_\theta(x) = \theta^T x = \theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$

Parameters: $\theta_0, \theta_1, \ldots, \theta_n$

Cost function:

$$J(\theta_0, \theta_1, \ldots, \theta_n) = \frac{1}{2m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)})^2$$

Gradient Descent

Previously (n=1):

Repeat $\{$

$$\theta_0 := \theta_0 - \alpha \underbrace{\frac{1}{m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})}_{\frac{\partial}{\partial\theta_0}J(\theta)}$$

$$\theta_1 := \theta_1 - \alpha \frac{1}{m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})x^{(i)}$$

(simultaneously update θ_0, θ_1)

$\}$

Gradient Descent

New algorithm $(n \geq 1)$:

Repeat $\{$

$$\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

(simultaneously update θ_j for $j = 0, \ldots, n$)

$\}$

Feature scaling

- The idea of feature scaling is all about making sure that all features are on the same scale

E.g. x_1 = size (0-2000 feet^2)

x_2 = number of bedrooms (1-5)

$$x_1 = \frac{\text{size (feet}^2\text{)}}{2000}$$

$$x_2 = \frac{\text{number of bedrooms}}{5}$$

- Generally get every feature in rang -1<= X <=1

Mean normalization

Replace x_i with $x_i - \mu_i$ to make features have approximately zero mean (Do not apply to $x_0 = 1$).

E.g. $x_1 = \frac{size - 1000}{2000}$

$x_2 = \frac{\#bedrooms - 2}{5}$

$$-0.5 \leq x_1 \leq 0.5, -0.5 \leq x_2 \leq 0.5$$

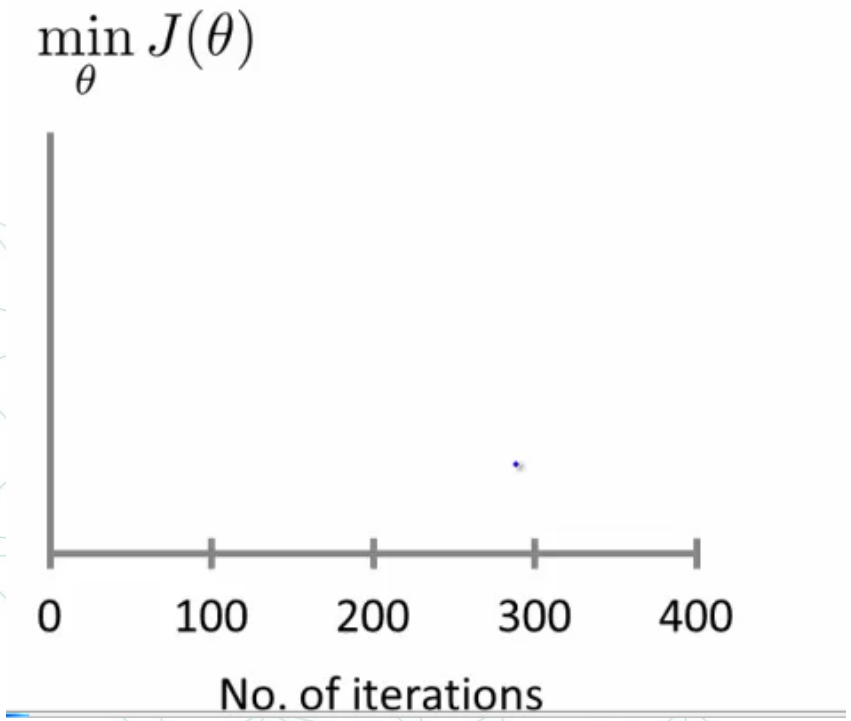
Gradient Descent optimization

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

"Debugging": How to make sure gradient descent is working correctly.

How to choose learning rate α.

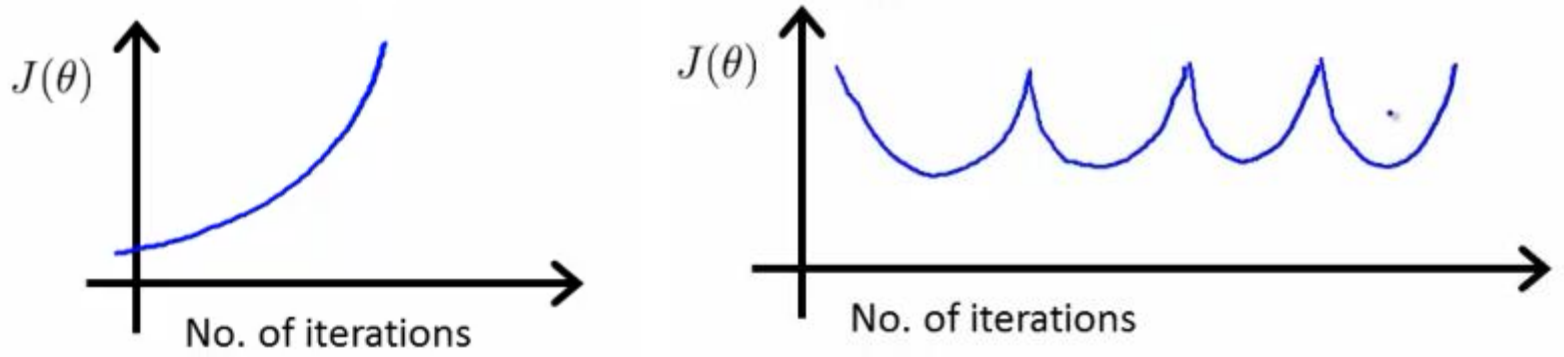
Convergence



Example automatic convergence test:

Declare convergence if $J(\theta)$ decreases by less than 10^{-3} in one iteration.

Making sure of gradient descent



- For sufficiently small α, $J(\theta)$ should decrease on every iteration.
- But if α is too small, gradient descent can be slow to converge.

Summary

- If α is too small: slow convergence.
- If α is too large: $J(\theta)$ may not decrease on every iteration; may not converge.

To choose α, try

$$\ldots, 0.001, \quad , 0.01, \quad , 0.1, \quad , 1, \ldots$$

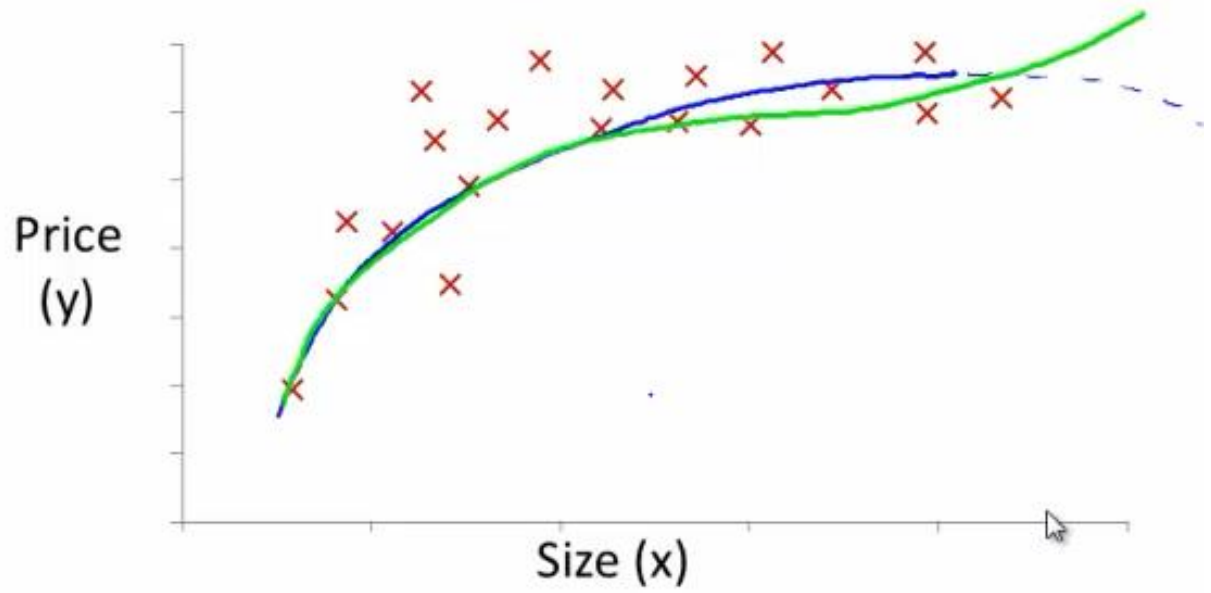
Polynomial regression

Housing prices prediction

$$h_\theta(x) = \theta_0 + \theta_1 \times frontage + \theta_2 \times depth$$



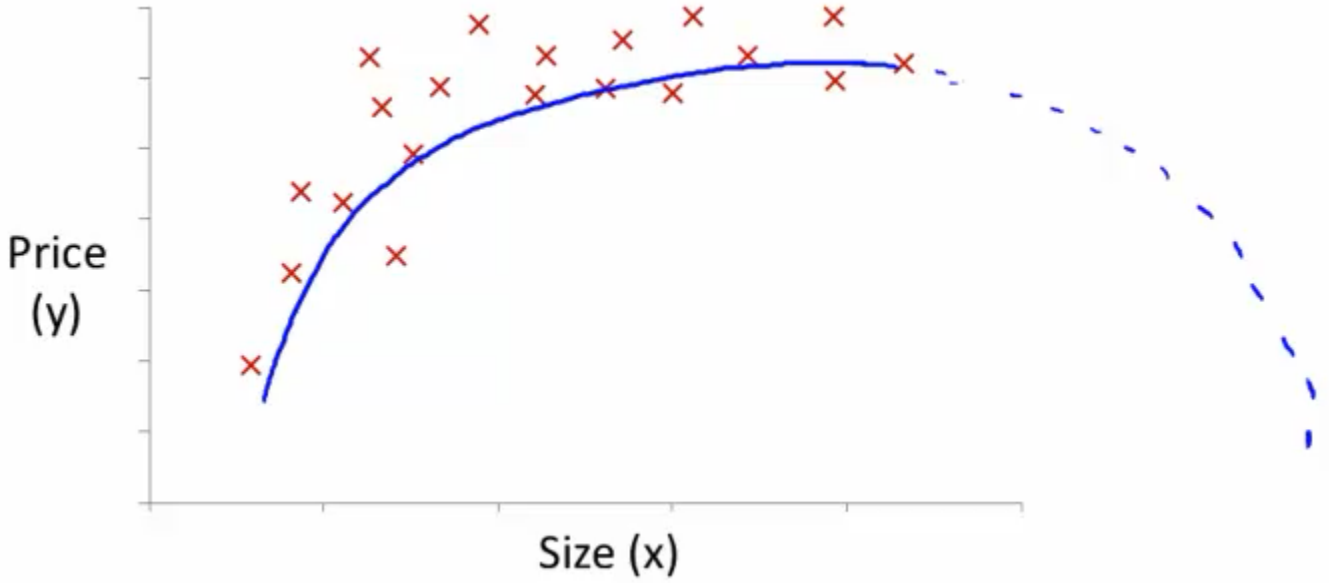
Polynomial regression



$$h_\theta(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3$$
$$= \theta_0 + \theta_1(size) + \theta_2(size)^2 + \theta_3(size)^3$$

$$x_1 = (size)$$
$$x_2 = (size)^2$$
$$x_3 = (size)^3$$

Choice of feature



$$h_\theta(x) = \theta_0 + \theta_1(size) + \theta_2(size)^2$$

$$h_\theta(x) = \theta_0 + \theta_1(size) + \theta_2\sqrt{(size)}$$

Q&A

Questions and answers

Logistic regression / classification

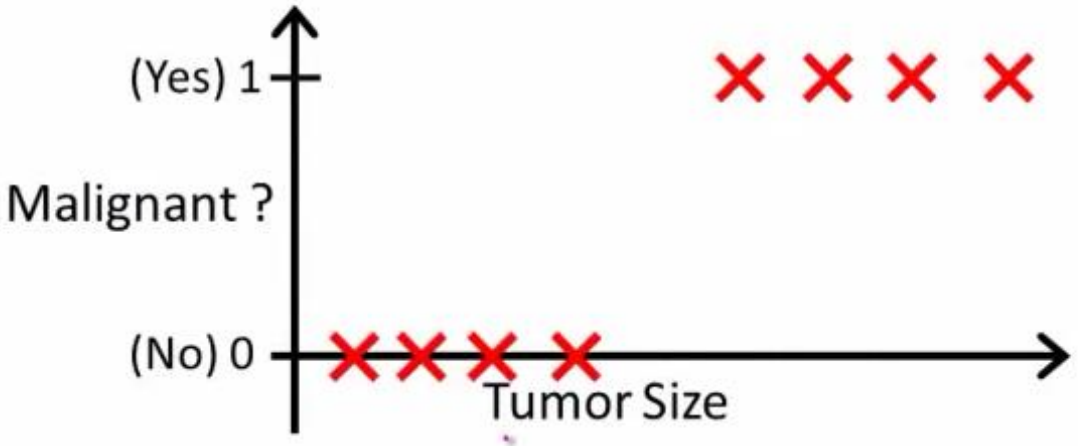
classification

Email: Spam / Non Spam
Online transactions: Fraud / Not Fraud
Tumor: Malignant / Benign ?

$y \in \{0, 1\}$

0: “Negative Class” (e.g., benign tumor)
1: “Positive Class” (e.g., malignant tumor)

Tumor



Threshold classifier output $h_\theta(x)$ at 0.5:

If $h_\theta(x) \geq 0.5$, predict "y = 1"

If $h_\theta(x) < 0.5$, predict "y = 0"

Hypothesis

Logistic Regression: $0 \leq h_\theta(x) \leq 1$

Logistic regression model

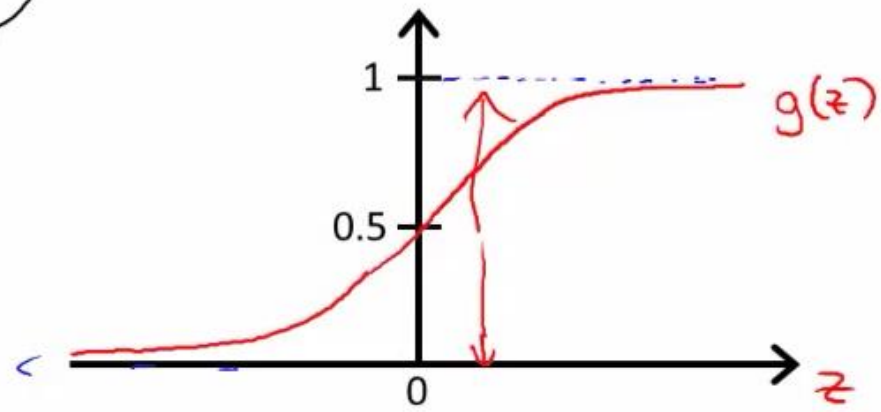
Want $0 \leq h_\theta(x) \leq 1$

$h_\theta(x) = g(\theta^T x)$

$g(z) = \frac{1}{1+e^{-z}}$

$z = \theta^T x$

$h_\theta(x) = \frac{1}{1+e^{-\theta^T x}}$



Sigmoid function

Logistic function

Interpreting hypothesis output

our hypothesis ($h_\theta(x)$) outputs a number, we treat that value as the estimated probability that y=1 on input x

Example

If X is a feature vector with $x_0 = 1$ (as always) and x_1 = tumourSize

$h_\theta(x) = 0.7$

Tells a patient they have a 70% chance of a tumor being malignant

We can write this using the following notation

$h_\theta(x) = P(y=1|x\ ;\ \theta)$

What does this mean?

Probability that y=1, given x, parameterized by θ

Since this is a binary classification task we know y = 0 or 1

So the following must be true

$P(y=1|x\ ;\ \theta) + P(y=0|x\ ;\ \theta) = 1$

$P(y=0|x\ ;\ \theta) = 1 - P(y=1|x\ ;\ \theta)$

Decision boundary

Gives a better sense of what the hypothesis function is computing

Better understand of what the hypothesis function looks like

One way of using the sigmoid function is;

When the probability of y being 1 is greater than 0.5 then we can predict y = 1

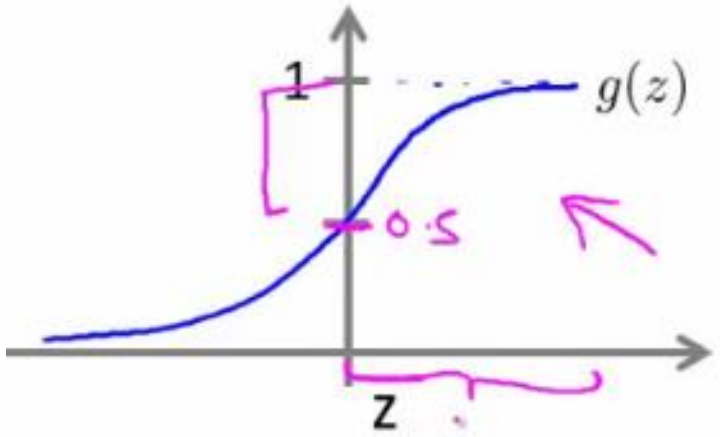
Else we predict y = 0

When is it exactly that $h_\theta(x)$ is greater than 0.5?

Look at sigmoid function

g(z) is greater than or equal to 0.5 when z is greater than or equal to 0

Decision boundary



So if z is positive, g(z) is greater than 0.5

$z = (\theta^T x)$

So when

$\theta^T x >= 0$

Then $h_\theta >= 0.5$

- So what we've shown is that the hypothesis predicts y = 1 when $\theta^T x >= 0$
  The corollary of that when $\theta^T x <= 0$ then the hypothesis predicts y = 0
  Let's use this to better understand how the hypothesis makes its predictions

.

Decision boundary

$h\theta(x) = g(\theta 0 + \theta 1x1 + \theta 2x2)$

So, for example

$\theta_0 = -3$

$\theta_1 = 1$

$\theta_2 = 1$

So our parameter vector is a column vector with the above values

So, θ^T is a row vector = [-3,1,1]

What does this mean?

The z here becomes $\theta^T x$

We predict "y = 1" if

$-3x_0 + 1x_1 + 1x_2 >= 0$

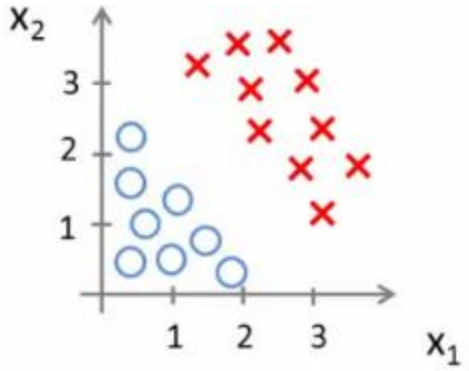
$-3 + x_1 + x_2 >= 0$

We can also re-write this as

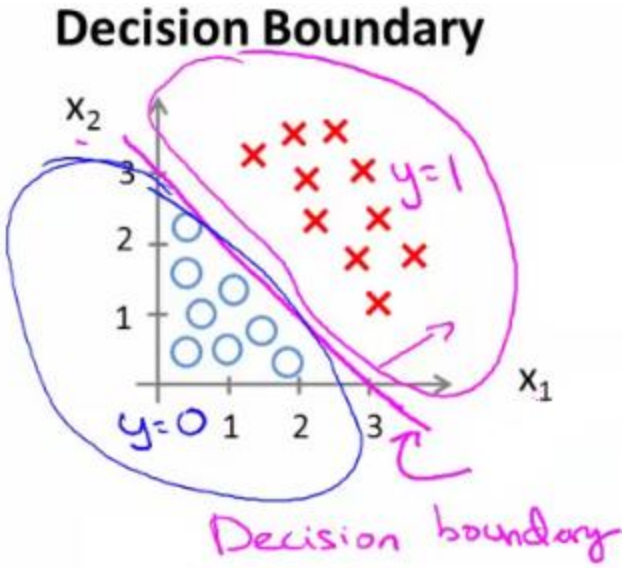
If $(x_1 + x_2 >= 3)$ then we predict y = 1

If we plot

$x_1 + x_2 = 3$ we graphically plot our **decision boundary**



Decision boundary



Nonlinear decision boundaries

Get logistic regression to fit a complex non-linear data set

Like polynomial regress add higher order terms

So say we have

$h_\theta(x) = g(\theta_0 + \theta_1 x_1 + \theta_3 x_1^2 + \theta_4 x_2^2)$

We take the transpose of the θ vector times the input vector

Say θ^T was [-1,0,0,1,1] then we say;

Predict that "y = 1" *if*

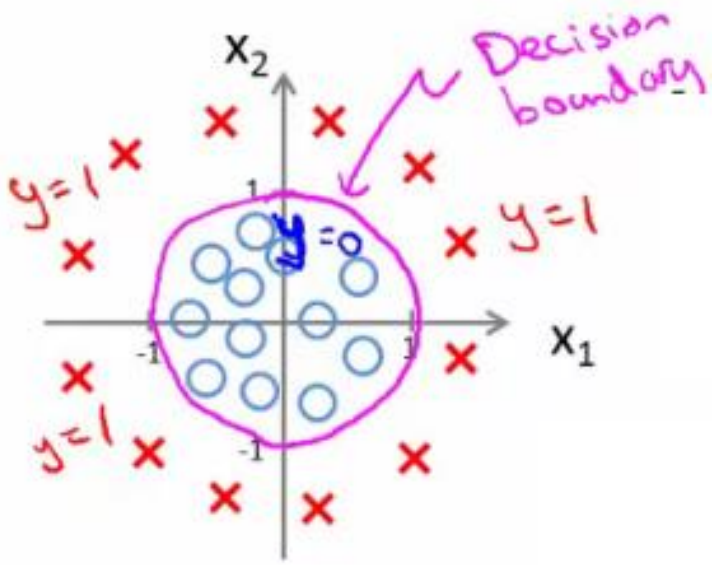
$-1 + x_1^2 + x_2^2 >= 0$

or

$x_1^2 + x_2^2 >= 1$

If we plot $x_1^2 + x_2^2 = 1$

This gives us a circle with a radius of 1 around 0



Cost function

Training set: $\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \cdots, (x^{(m)}, y^{(m)})\}$

m examples $x \in \begin{bmatrix} x_0 \\ x_1 \\ \cdots \\ x_n \end{bmatrix}$ $x_0 = 1, y \in \{0, 1\}$

$$h_\theta(x) = \frac{1}{1 + e^{-\theta^T x}}$$

Cost function

This is the situation

Set of m training examples

Each example is a feature vector which is n+1 dimensional

$x_0 = 1$

$y \in \{0,1\}$

Hypothesis is based on parameters (θ)

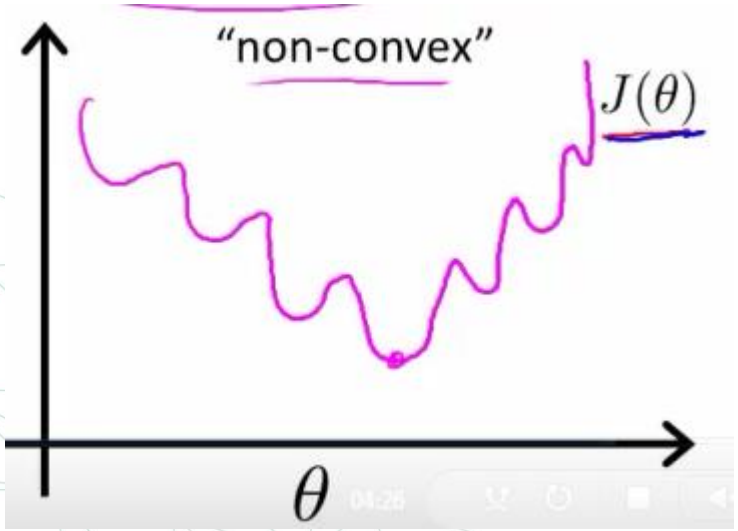
Given the training set how to we chose/fit θ?

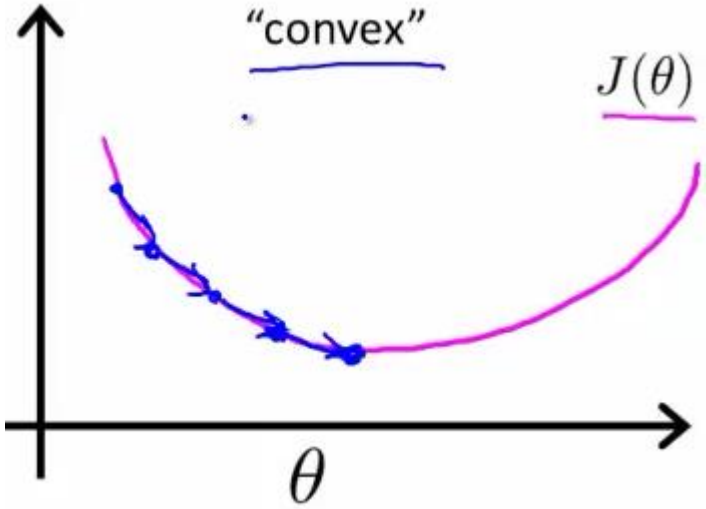
Linear regression uses the following function to determine θ

$$J(\theta) = \frac{1}{m} \sum_{i=1}^{m} \frac{1}{2} \left(h_\theta(x^{(i)}) - y^{(i)} \right)^2$$

$$J(\theta) = \frac{1}{m} \sum_{i=1}^{m} \mathrm{Cost}(h_\theta(x), y)$$

Cost function



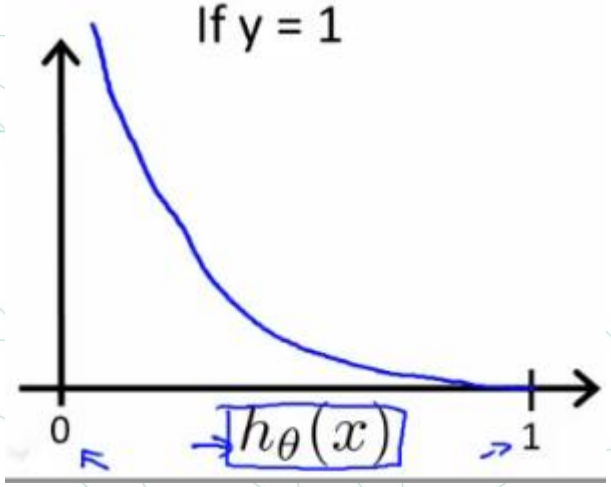


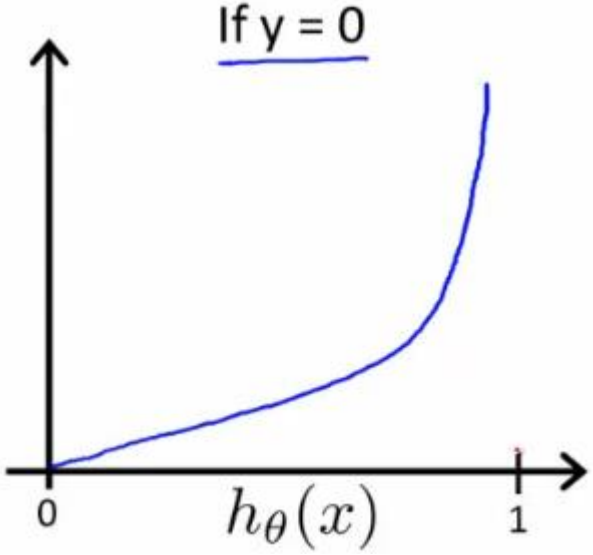
Simplified Cost Function

$$J(\theta) = \frac{1}{m} \sum_{i=1}^{m} \text{Cost}(h_\theta(x^{(i)}), y^{(i)})$$

$$\text{Cost}(h_\theta(x), y) = \begin{cases} -\log(h_\theta(x)) & \text{if } y = 1 \\ -\log(1 - h_\theta(x)) & \text{if } y = 0 \end{cases}$$

Note: $y = 0$ or 1 always





Simplified cost function

- Can write cost function is
  - **cost(h_θ (x),y) = -ylog(h_θ(x)) - (1-y)log(1- h_θ(x))**
    - This equation is a more compact of the two cases above
- We know that there are only two possible cases
  - y = 1
    - Then our equation simplifies to
      - -log(h_θ(x)) - (0)log(1 - h_θ(x))
        - -log(h_θ(x))
        - Which is what we had before when y = 1
  - y = 0
    - Then our equation simplifies to
      - -(0)log(h_θ(x)) - (1)log(1 - h_θ(x))
      - = -log(1- h_θ(x))
      - Which is what we had before when y = 0
  - Clever!

.

In summary

$$J(\theta) = -\frac{1}{m}[\sum_{i=1}^{m} y^{(i)} \log h_\theta(x^{(i)}) + (1 - y^{(i)}) \log (1 - h_\theta(x^{(i)}))]$$

To fit parameters θ :

$$\min_\theta J(\theta)$$

Get θ

To make a prediction given new x:

Output $h_\theta(x) = \frac{1}{1+e^{-\theta^T x}}$

$p(y=1 \mid x;\theta)$

Gradient descent

Repeat {

$$\theta_j := \theta_j - \alpha \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

(simultaneously update all θ_j)

}

Repeatedly update each parameter using a learning rate

- If you had *n* features, you would have an n+1 column vector for θ
- This equation is the same as the linear regression rule
  The only difference is that our definition for the hypothesis has changed
- Previously, we spoke about how to monitor gradient descent to check it's working
  Can do the same thing here for logistic regression
- When implementing logistic regression with gradient descent, we have to update all the θ values (θ_0 to θ_n) simultaneously
  Could use a for loop
  Better would be a vectorized implementation
- Feature scaling for gradient descent for logistic regression also applies here

Decision Tree

Introduction

A classification scheme which generates a set of rules from given data set.

The set of records available for developing classification methods is divided into two , Test and Training

The attributes of the records are categories into two types:

- Numerical
- Categorical

Introduction

- A decision tree is a tree with the following properties
  - An inner node represents an attribute.
  - An edge represents a test on the attribute
  - A leaf represents one of the classes.
- Construction of a decision tree
  - Based on training data
  - Top-Down strategy

Decision Tree example

- The dataset has five attributes
- This is a special attribute: the

  attribute class is the class label

- There are attributes numerical
- Categorical attributes can not

Be ordered

OUTLOOK	TEMP(F)	HUMIDITY(%)	WINDY	CLASS
sunny	79	90	true	no play
sunny	56	70	false	play
sunny	79	75	true	play
sunny	60	90	true	no play
overcast	88	88	false	no play
overcast	63	75	true	play
overcast	88	95	false	play
rain	78	60	false	play
rain	66	70	false	no play
rain	68	60	true	no play

- We want to know the set of the rules to determine the play of the golf

Decision Tree example

- We have five leaf nodes
- Each node represents a rule
- We have the following rules in the tree

RULE 1	If it is sunny and the humidity is not above 75%, then play.
RULE 2	If it is sunny and the humidity is above 75%, then do not play.
RULE 3	If it is overcast, then play.
RULE 4	If it is rainy and not windy, then play.
RULE 5	If it is rainy and windy, then don't play.

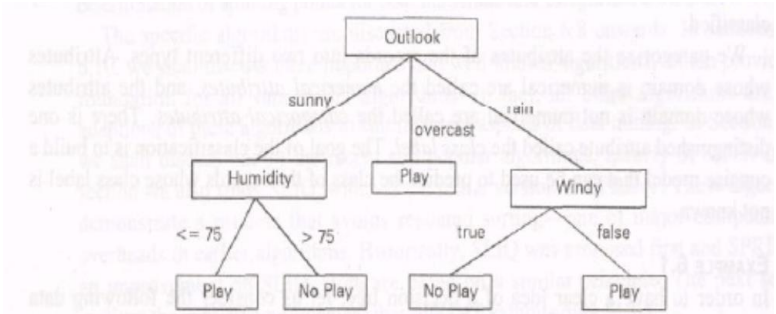


Figure 6.1 A Decision Tree

Classification

- The classification of an unknown input vector is done by traversing the tree from the root node to a leaf node
- A sample enters at a root anode
- At the root a test is applied to determine which node the record will encounter next
- This process is repeated until the record arrives at leaf node
- The path is a rule which is used to classify the records

Classification

RULE 1

If it is sunny and the humidity is not above 75%, then play.

Table 6.2 Test Data Set

OUTLOOK	TEMP(F)	HUMIDITY(%)	WINDY	CLASS
sunny	79	90	true	play
sunny	56	70	false	play
sunny	79	75	true	no play
sunny	60	90	true	no play
overcast	88	88	false	no play
overcast	63	75	true	play
overcast	88	95	false	play
rain	78	60	false	play
rain	66	70	false	no play
rain	68	60	true	play

Categorical Attributes

- Consider the dataset
- There are three attributes
- Class is the class label
- Pincode is a categorical one
- No useful information in pin code

Table 6.3 Another Example

ID	AGE	PINCODE	CLASS
1	30	5600046	C1
2	25	5600046	C1
3	21	5600023	C2
4	43	5600046	C1
5	18	5600023	C2
6	33	5600023	C1
7	29	5600023	C1
8	55	5600046	C2
9	48	5600046	C1

Categorical data

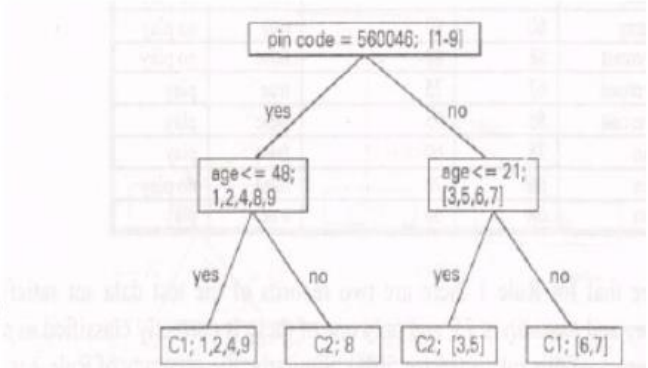


Figure 6.2 A Decision Tree

At root level, we have 9 records. The associated splitting criterion is *pincode* = 500 046.

As a result, we split the records into two subsets. Records 1, 2, 4, 8, and 9 are to the left child note and remaining to the right node.

The process is repeated at every node.

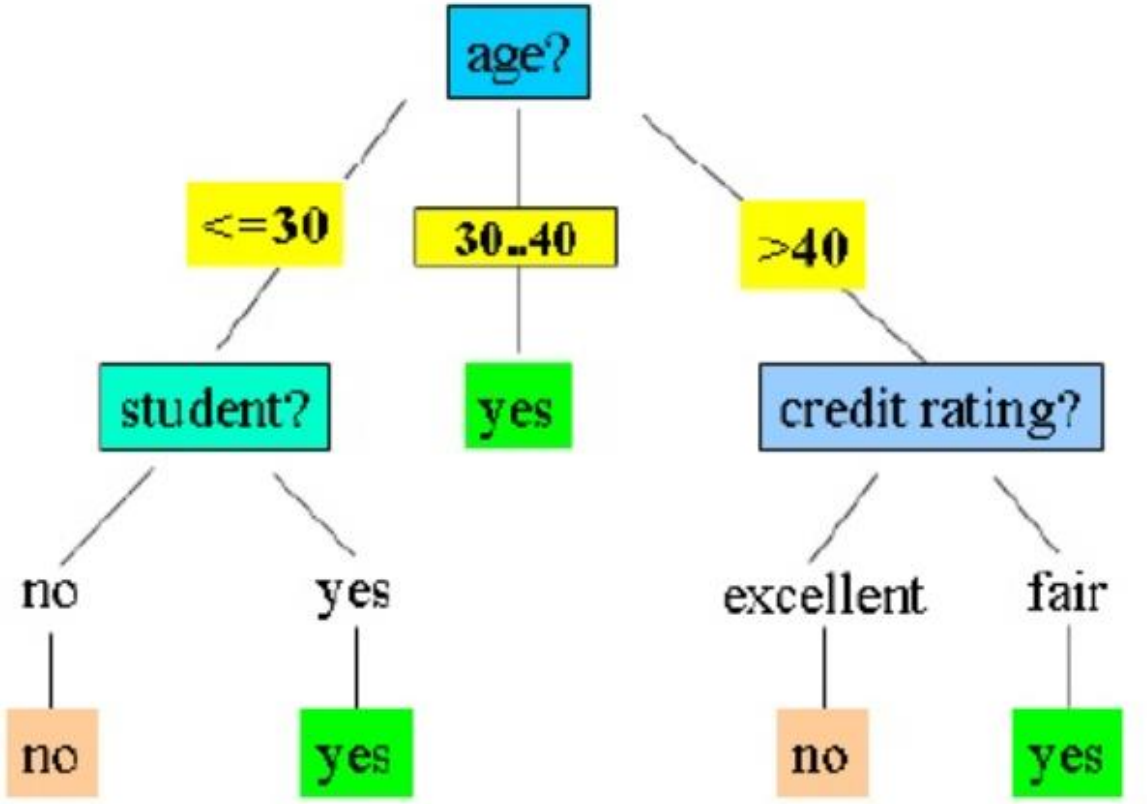
Pros and Cons of Decision Tree

- Pros
  - Generate understandable rules
  - Able to handle both numerical and categorical
  - Indicate which fields are the most important
- Cons
  - Computationally expensive
  - Each feature gets examined
  - Some times can deal with only binary target classes

Quinlan's ID3

age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31…40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31…40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31…40	medium	no	excellent	yes
31…40	high	yes	fair	yes
>40	medium	no	excellent	no

Example


age?
<=30
30..40
>40
student?
yes
credit rating?
no
yes
excellent
fair
no
yes
no
yes

Solution

IF *age* = "<=30" AND *student* = "*no*" THEN *buys_computer* = "*no*"

IF *age* = "<=30" AND *student* = "*yes*" THEN *buys_computer* = "*yes*"

IF *age* = "31…40" THEN *buys_computer* = "*yes*"

IF *age* = ">40" AND *credit_rating* = "*excellent*" THEN *buys_computer* = "*yes*"

IF *age* = "<=30" AND *credit_rating* = "*fair*" THEN *buys_computer* = "*no*"

Algorithm for Decision Tree

- Basic algorithm (a greedy algorithm)
  - Tree is constructed in a top-down recursive divide-and-conquer manner
  - At start, all the training examples are at the root
  - Attributes are categorical (if continuous-valued, they are discretized in advance)
  - Examples are partitioned recursively based on selected attributes
  - Test attributes are selected on the basis of a heuristic or statistical measure (e.g., information gain)
- Conditions for stopping partitioning
  - All samples for a given node belong to the same class
  - There are no remaining attributes for further partitioning – majority voting is employed for classifying the leaf
  - There are no samples left

Attribute Selection: Info Gain

- Select the one with the highest information gain
- Information required to any class

$$I(s_1, s_2, \ldots, s_m) = -\sum_{i=1}^{m} \frac{s_i}{s} \log_2 \frac{s_i}{s}$$

- Entropy Measure

$$E(A) = \sum_{j=1}^{v} \frac{s_{1j} + \ldots + s_{mj}}{s} I(s_{1j}, \ldots, s_{mj})$$

- Information gained by branching

$$Gain(A) = I(s_1, s_2, \ldots, s_m) - E(A)$$

Entropy

- Entropy measures the homogeneity (purity) of a set of examples.
- It gives the information content of the set in terms of the class labels of the examples.
- Consider that you have a set of examples, S with two classes, P and N. Let the set have p instances for the class P and n instances for the class N.
- So the total number of instances we have is t = p + n. The view [p, n] can be seen as a class distribution of S.

The entropy for S is defined as

- $\text{Entropy}(S) = - (p/t).\log_2(p/t) - (n/t).\log_2(n/t)$

- Example: Let a set of examples consists of 9 instances for class positive, and 5 instances for class negative.
- Answer: p = 9 and n = 5.
- So $\text{Entropy}(S) = - (9/14).\log_2(9/14) - (5/14).\log_2(5/14)$
- $= -(0.64286)(-0.6375) - (0.35714)(-1.48557)$
- $= (0.40982) + (0.53056)$
- $= 0.940$

age	p_i	n_i	$I(p_i, n_i)$
<=30	2	3	0.971
30…40	4	0	0
>40	3	2	0.971

age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31…40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31…40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31…40	medium	no	excellent	yes
31…40	high	yes	fair	yes
>40	medium	no	excellent	no

R. Ak

Example

$$E(age) = \frac{5}{14} I(2,3) + \frac{4}{14} I(4,0) + \frac{5}{14} I(3,2) = 0.694$$

$\frac{5}{14} I(2,3)$ means "age <=30" has 5 out of 14 samples, with 2 yes's and 3 no's. Hence

$$Gain(age) = I(p,n) - E(age) = 0.246$$

Similarly,

$$Gain(income) = 0.029$$
$$Gain(student) = 0.151$$
$$Gain(credit_rating) = 0.048$$

Gini Index

- All attributes considered continuous valued
- Assume several possible split
- May need other tools to split i.e. clustering
- Can be modified for categorical

Gini Index

If a data set T contains examples from n classes, gini index, $gini(T)$ is defined as

$$gini\ (T) = 1 - \sum_{j=1}^{n} p_j^2$$

where p_j is the relative frequency of class j in T.

If a data set T is split into two subsets T_1 and T_2 with sizes N_1 and N_2 respectively, the *gini* index of the split data contains examples from n classes, the *gini* index $gini(T)$ is defined as

$$gini_{split}(T) = \frac{N_1}{N} gini(T_1) + \frac{N_2}{N} gini(T_2)$$

The attribute provides the smallest $gini_{split}(T)$ is chosen to split the node (*need to enumerate all possible splitting points for each attribute*).

رواد مصر الرقمية

وزارة الاتصـــــالات
وتكنولوجيا المعلومات

Q&A

Questions and answers

Random Forest

Introduction

- An ensemble classifier using many decision tree models.
- Can be used for classification or regression
- Accuracy and variable importance info are represented with the results

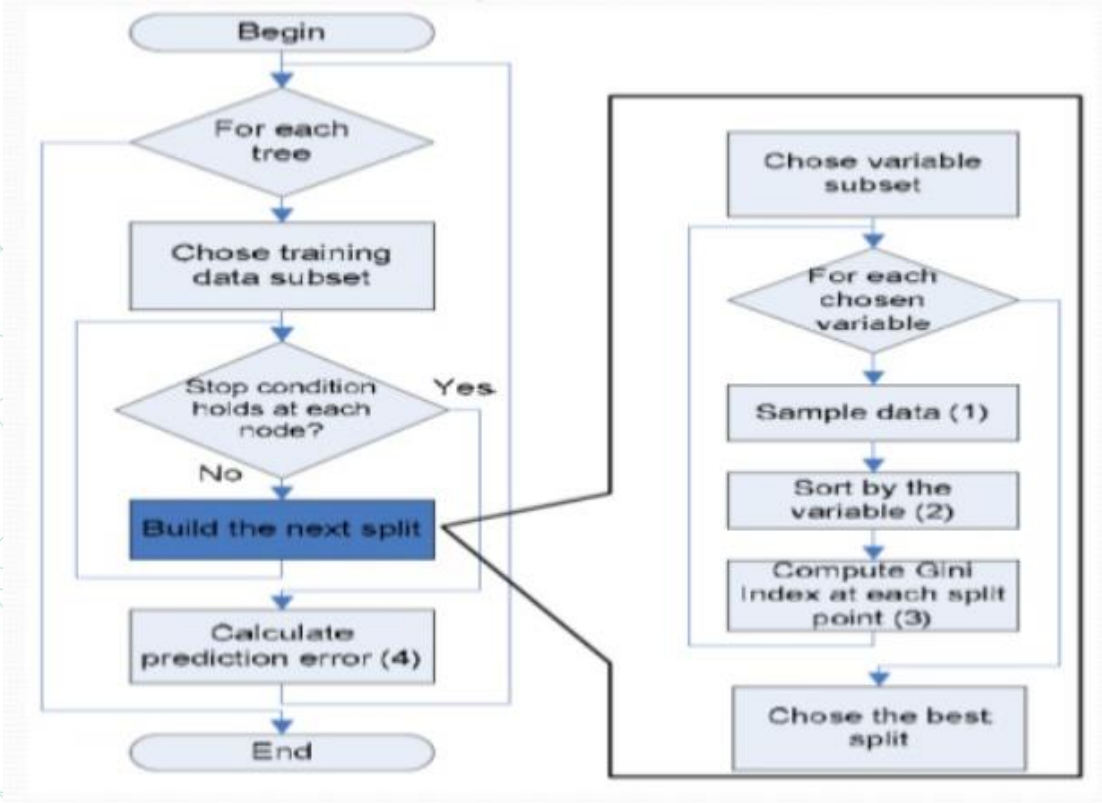
The Algorithm

- You have a dataset with M samples and N features
- M number should be large
- Choose a training set and testing set to validate the Tree Model accuracy
- For each node a the tree choose the best split based on the measuring criteria (GINI, Entropy)
- Each tree is fully grown and not pruned

The Algorithm

- A random forest is classifier consisting of a collection of tree structured classifiers K trees
- Each tree casts a unit vote determining the last class of the input
- The final class from each tree is aggregated and voted by weighted values to construct the final classifier

Flow chart


Begin
For each tree
Chose training data subset
Stop condition holds at each node?
Yes
No
Build the next split
Calculate prediction error (4)
End
Chose variable subset
For each chosen variable
Sample data (1)
Sort by the variable (2)
Compute Gini Index at each split point (3)
Chose the best split

Pros

- It produces a highly accurate classifier and learning is fast
- It runs efficiently on large data bases
- It can handle thousand of input variables
- It creates proximity between samples which may be used to clustering , locating outliers or give views of data
- It offers an experimental method for detecting variable interactions.

Cons

- Requires much computational power
- Requires much processing time
- Suffers from lack of interpretability

Operation of Random Forest

- A random seed is chosen which pulls out a random sample from the training dataset
- With the selected dataset, a random set of attributes are chosen based on user defined values
- Not all the input variables are considered to prevent any overfitting
- For N features only R are chosen as attributes for each tree (R subset of N)

Operation of Random Forest

- Create the best possible split from this set
- The process repeats until termination conditions are reached, leaves are too small to split

Example

RECORD	ATTRIBUTES		CLASS
	HOME_TYPE	SALARY	
1	31	3	1
2	30	1	0
3	6	2	0
4	15	4	1
5	10	4	0

Example

Partitions after the Binary Split on HOME_TYPE <=6 by the Random Forest

Attribute	Number of records		
	Zero(0)	One (1)	N = 5
HOME_TYPE <=6	1	0	n1 = 1
HOME_TYPE > 6	2	2	n2 = 4

Then Gini(D1) , Gini (D2) , and $Gini_{SPLIT}$ are calculated as follows:

Example

Then $Gini(D_1)$, $Gini(D_2)$, and $Gini_{SPLIT}$ are calculated as follows:

$$Gini(HOME_TYPE \le 6) = 1 - (1^2 + 0^2) = 0$$

$$Gini(HOME_TYPE > 6) = 1 - \left(\left(\frac{2}{4}\right)^2 + \left(\frac{2}{4}\right)^2\right) = 0.5$$

$$Gini_{SPLIT} = \left(\frac{1}{5}\right) \times 0 + \left(\frac{4}{5}\right) \times 0.5 = 0.4$$

Example

In the next step, the data set at HOME_TYPE <=10 is split and tabulated in Table.

Partitions after the Binary Split on HOME_TYPE <=10 by the Random Forest:

Attribute	Number of records		
	Zero(0)	One (1)	N = 5
HOME_TYPE <=10	2	0	n1 = 2
HOME_TYPE > 10	1	2	n2 = 3

Then Gini (D1) , Gini (D2) , and Gini $_{SPLIT}$ are calculated as follows:

Example

$$Gini(HOME_TYPE \le 10) = 1 - (1^2 + 0^2) = 0$$

$$Gini(HOME_TYPE > 10) = 1 - \left(\left(\frac{1}{3}\right)^2 + \left(\frac{2}{3}\right)^2\right) = 0.4452$$

$$Gini_{SPLIT} = \left(\frac{2}{5}\right) \times 0 + \left(\frac{3}{5}\right) \times 0.4452 = 0.2671$$

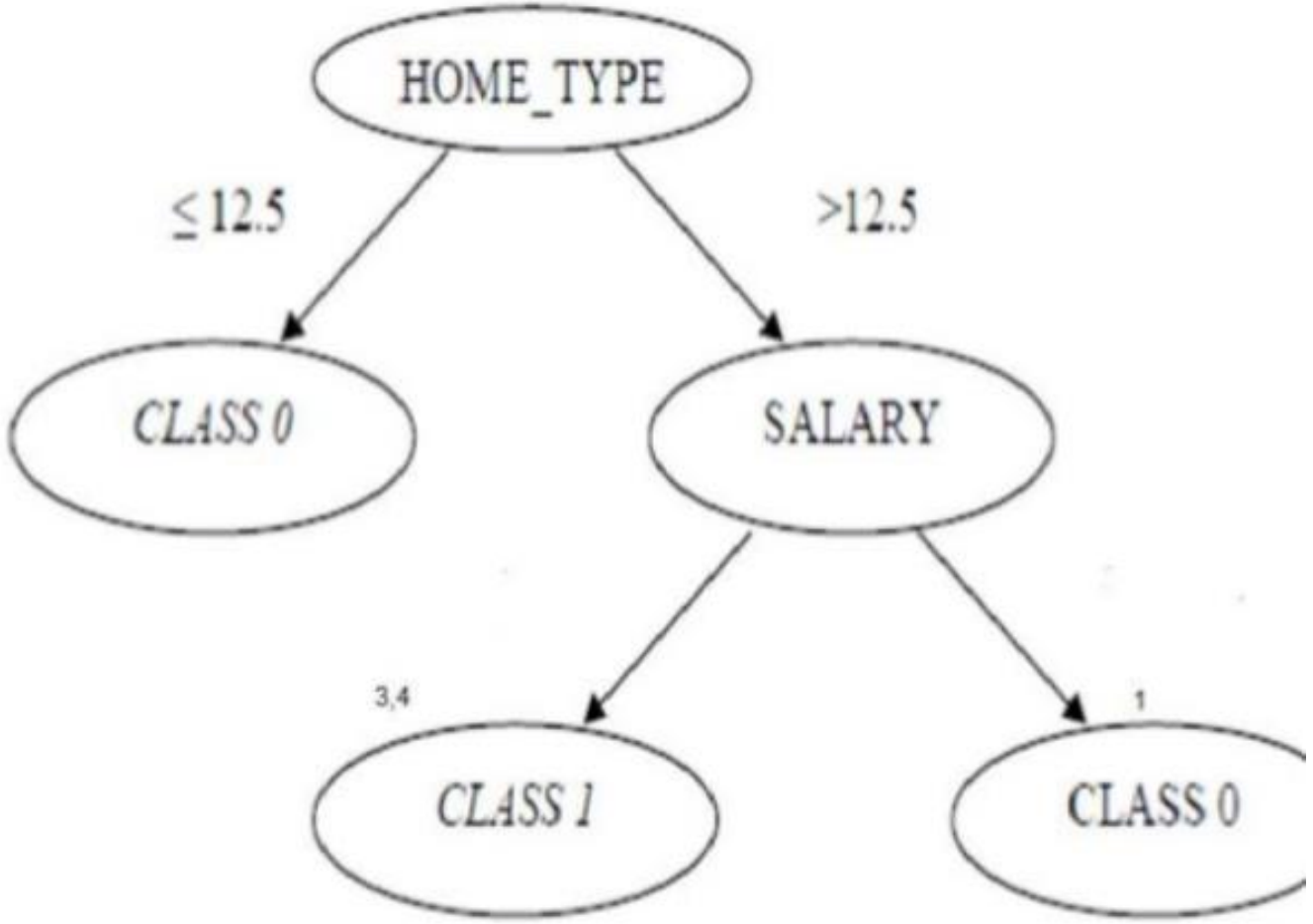
Example

tabulates the gini index value for the HOME_TYPE attribute at all possible splits.

Gini SPILT	Value
Gini SPLT(HOME_TYPE<=6)	0.4000
Gini SPLT(HOME_TYPE<=10)	0.2671
Gini SPLT(HOME_TYPE<=15)	0.4671
Gini SPLT(HOME_TYPE<=30)	0.3000
Gini SPLT(HOME_TYPE<=31)	0.4800

the split HOME_TYPE <= 10 has the lowest value

Example

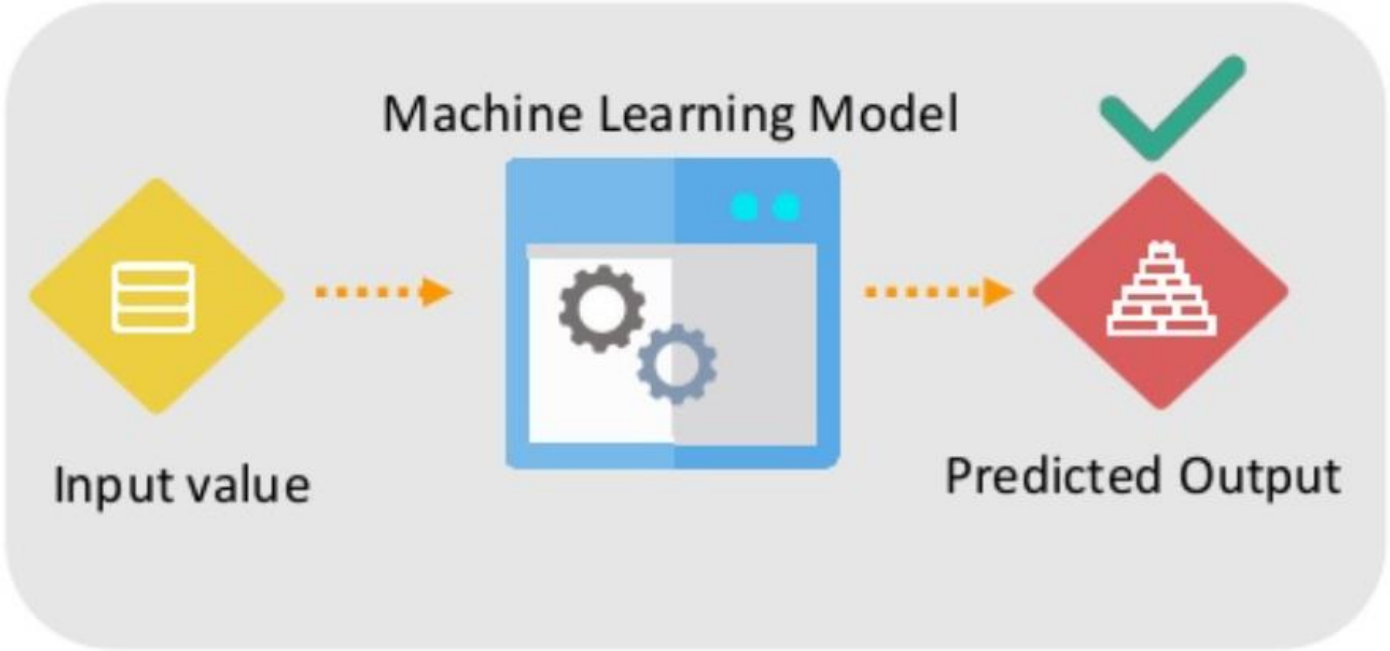


KNN

Introduction

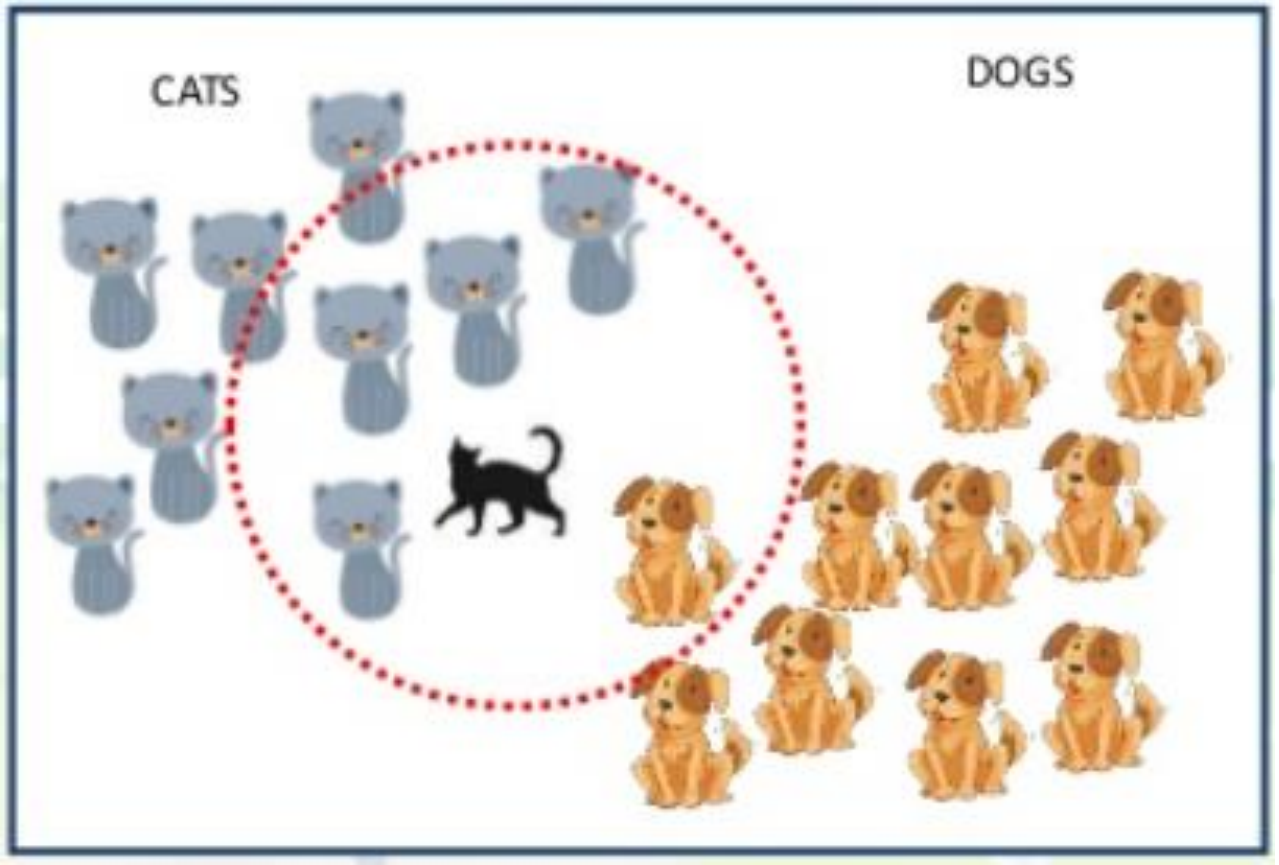
- Why do we need KNN?
- What is KNN?
- How do we choose the factor K ?
- When do we use KNN ?
- How does KNN algorithm Works ?
- Use Case

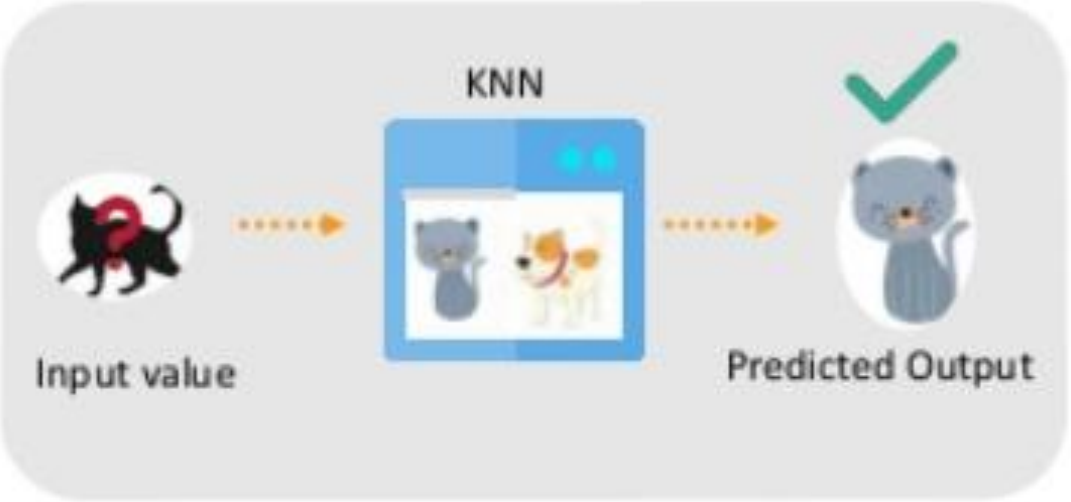
The Algorithm



Classification Based On Features

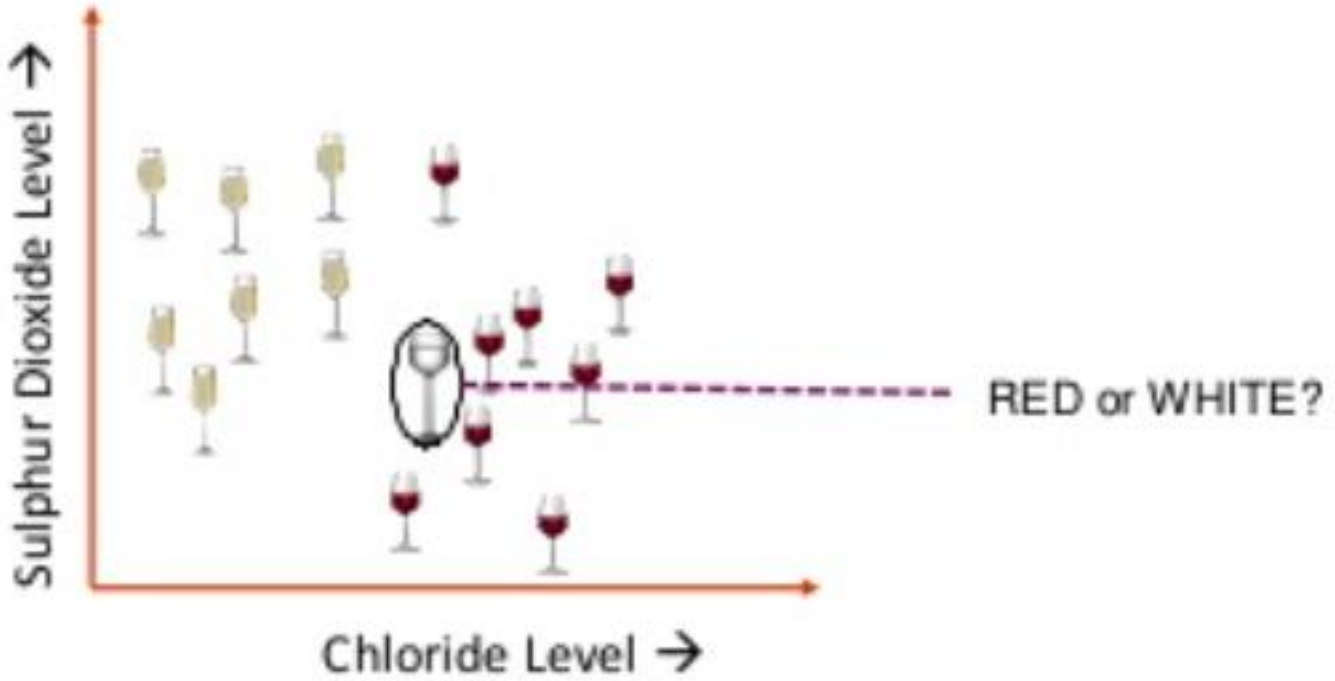
CATS	DOGS
	
Sharp Claws, uses to climb	Dull Claws
Smaller length of ears	Bigger length of ears
Meows and purrs	Barks
Doesn't love to play around	Loves to run around


CATS
DOGS

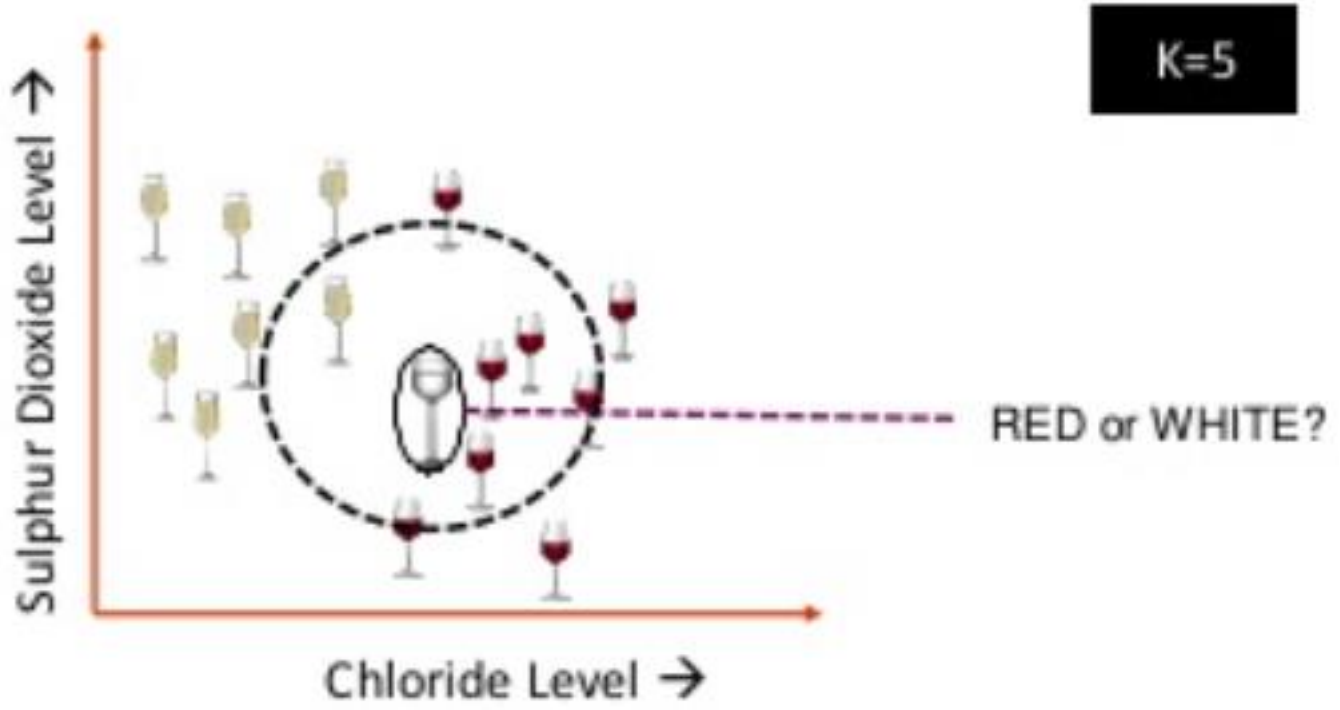

KNN
Input value
Predicted Output

What is KNN ?

- KNN - K nearest neighbors is one of the simplest supervised machine learning algorithms mostly Used for classification based on the K neighbors .


Sulphur Dioxide Level →
RED or WHITE?
Chloride Level →

What is K ?

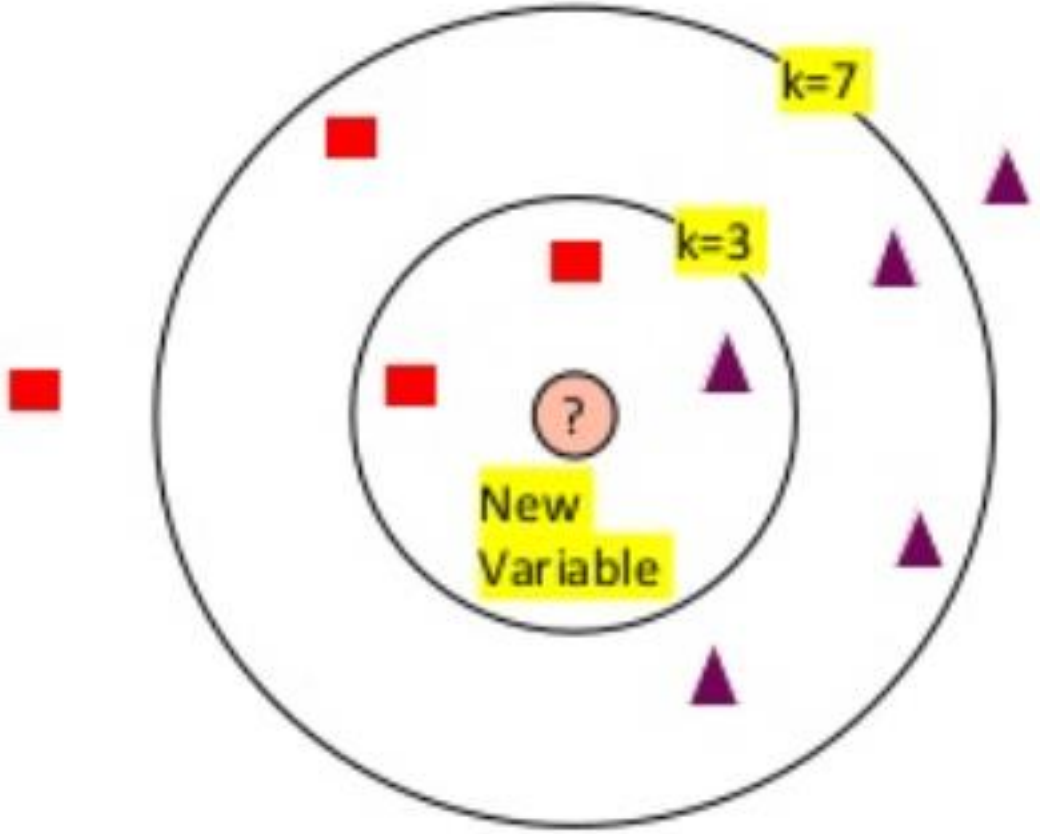


What is K ?

- K is a parameter in the KNN refers to the number of nearest neighbors to include in the majority voting process .

How Do we choose K ?

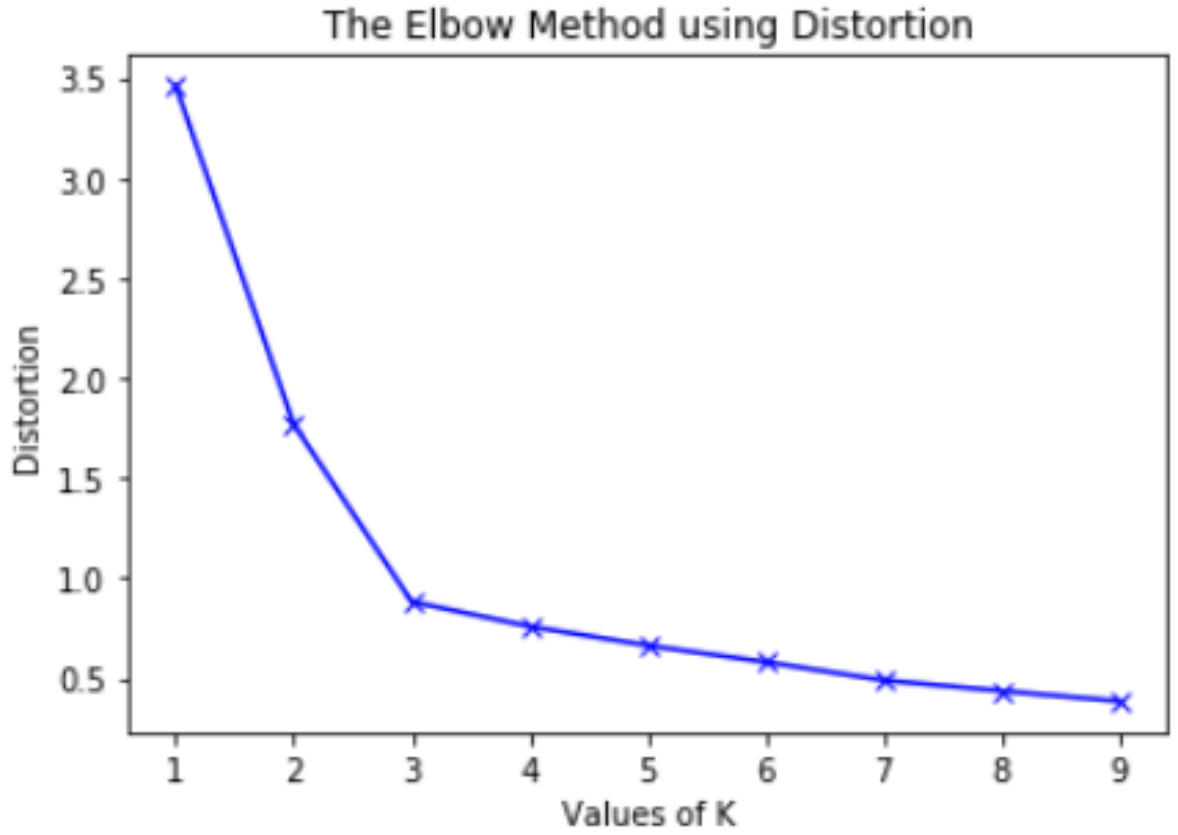
- KNN algorithm is based on features similarity choosing the right value of K is a process called parameter tuning and it is important for better accuracy .


k=7
k=3
?
New
Variable

How to choose K ?

- Sqrt(n), where n is the total number of data points
- Odd value of K is selected to avoid confusion between two classes of data

Elbow Method



When to Use KNN ?

- Data is labeled
- Data is noise free
- Data is small (as the KNN is a lazy learner “does not learn discriminative features”)

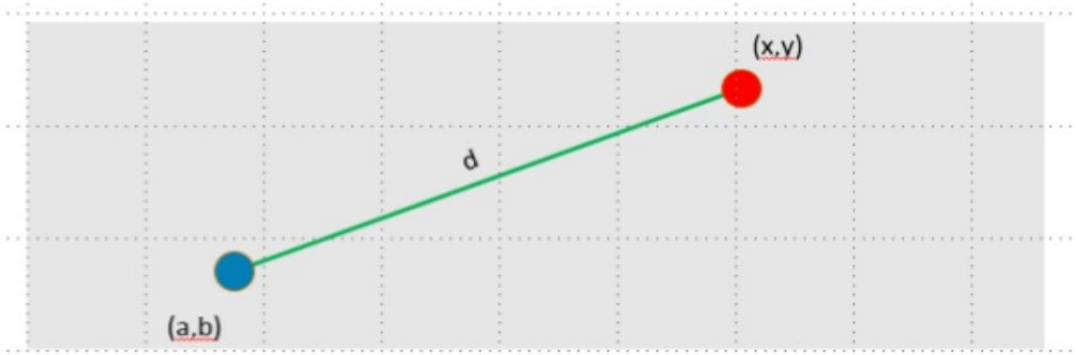
How does KNN algorithm Work ?

Weight(x2)	Height(y2)	Class
51	167	Underweight
62	182	Normal
69	176	Normal
64	173	Normal
65	172	Normal
56	174	Underweight
58	169	Normal
57	173	Normal
55	170	Normal

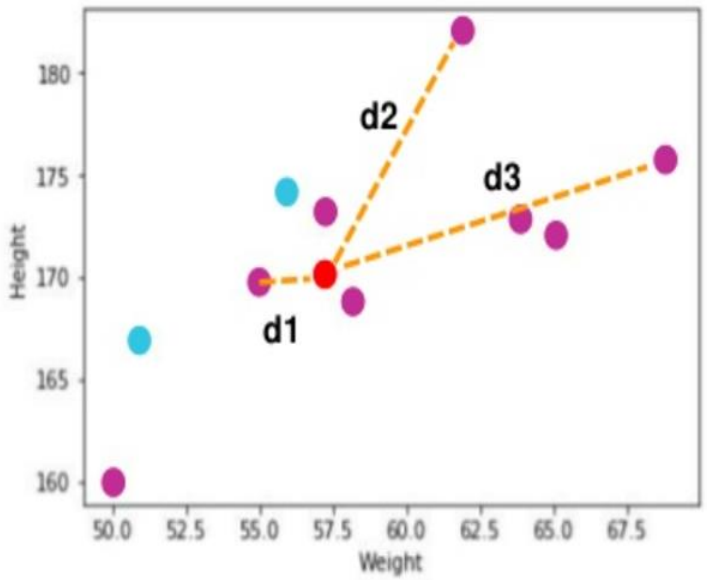
How does KNN algorithm Work ?

- Tos find the Nearest neighbors we will have to calculate the Euclidean Distance
- It is the distance between two points in plane

$$dist(d) = \sqrt{(x - a)^2 + (y - b)^2}$$



Let's calculate it to understand clearly:



Unknown data point

dist(**d1**)= $\sqrt{(170-167)^2 + (57-51)^2}$ ~= 6.7

dist(**d2**)= $\sqrt{(170-182)^2 + (57-62)^2}$ ~= 13

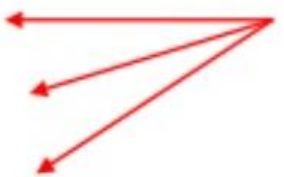
dist(**d3**)= $\sqrt{(170-176)^2 + (57-69)^2}$ ~= 13.4

Similarly, we will calculate Euclidean distance of unknown data point from all the points in the dataset

Where (x1, y1) = (57, 170) whose class we have to classify

Weight(x2)	Height(y2)	Class	Euclidean Distance
51	167	Underweight	6.7
62	182	Normal	13
69	176	Normal	13.4
64	173	Normal	7.6
65	172	Normal	8.2
56	174	Underweight	4.1
58	169	Normal	1.4
57	173	Normal	3
55	170	Normal	2

Weight(x2)	Height(y2)	Class	Euclidean Distance
51	167	Underweight	6.7
62	182	Normal	13
69	176	Normal	13.4
64	173	Normal	7.6
65	172	Normal	8.2
56	174	Underweight	4.1
58	169	Normal	1.4
57	173	Normal	3
55	170	Normal	2



k = 3

57 kg	170 cm	?

Conclusion

- A positive integer k is specified anlon with a new sample
- We select the K entries in out database which are closest to the new sample
- We find the most common classification of these entries
- This the classification we give to the new sample

Use Case

Diabetes Classification Dataset

Q&A

Questions and answers

Thanks